Harness Self-Evolution: Theory, Algorithms, Evaluation, Applications, and Impact

Chuyang Wei[1], **Haoxiang Guan**[1], **Minghang Zhu**[1], **Kexin Zhou**[1], **Jiyan He**[1,2], **Yifei Shen**[1,2,†]

[1]Zhongguancun Academy · [2]Zhongguancun Institute of Artificial Intelligence · [†]Corresponding author

Abstract

Building agents that learn from experience is a central goal of artificial intelligence. Harness self-evolution offers a path toward this goal by updating an agent's instructions, memory, workflows, and tools without changing model weights. In this article, we study how systems turn execution feedback into persistent harness changes, and we review the offline optimizers that many of them build on. We organize algorithms by their update targets, distinguishing instructions, memory, and workflow specifications interpreted by fixed components from executable tools and runtime logic, and compare how they use feedback to propose and admit changes. We relate harness search to evolutionary algorithms and use a continual PAC-learning perspective to state sufficient conditions under which an admission comparison certifies improvement. For evaluation, we separate four claims a reported gain can rest on: selection, whether the number was measured or picked by the search that produced the update; activation, whether the update operated in the scored runs; attribution, whether the difference came from the update rather than from run-to-run noise, extra inference compute, or a changed evaluator; and improvement, whether the difference holds away from the tasks that produced it. Read against these claims, the reviewed reports often evaluate on held-out tasks but seldom record whether the accepted update ran in the scored runs or separate its effect from noise and added compute. We examine how deployment settings change validation requirements and how persistent, shareable harness state may affect work, knowledge ownership, security, and governance. We conclude with open problems in optimization, infrastructure, evaluation, and responsible deployment. The project repository is available at https://github.com/zgcagi/Harness-RSI.

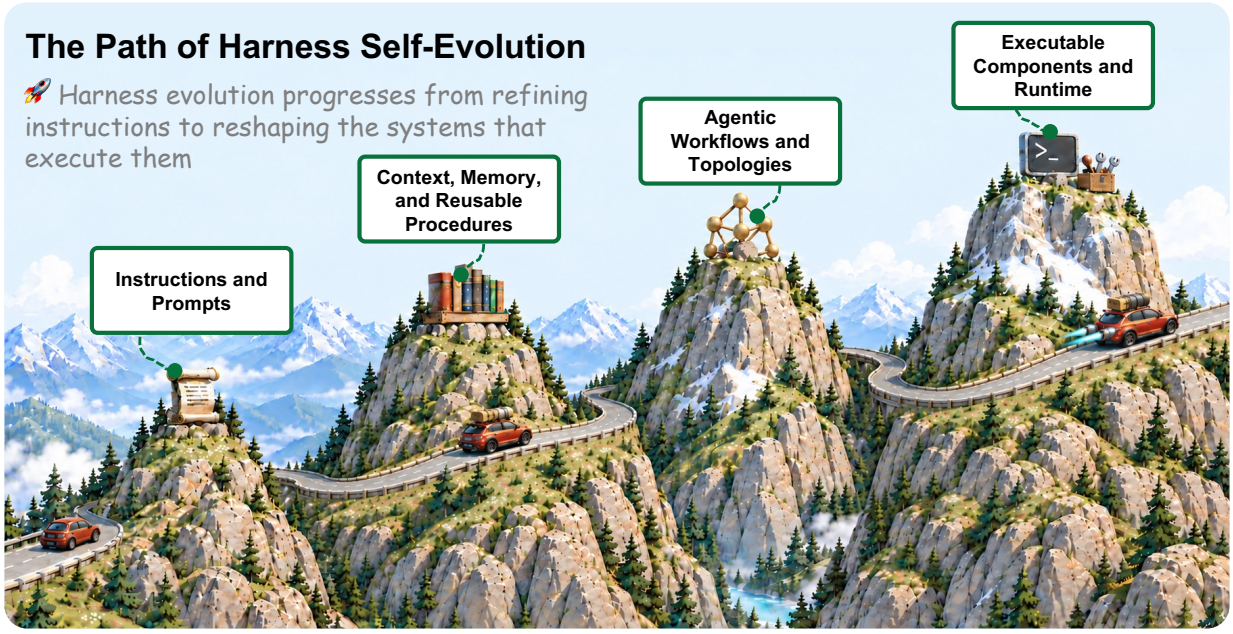


Figure 1 The path of harness self-evolution.

Contents

1 Introduction **3**

2 Definition of Harness Self-Evolution **6**

3 Interpreted Update Targets **8**
3.1 From Recursive Self-Improvement to Harness Adaptation 8
3.2 Instructions and Prompts 10
3.3 Context, Memory, and Reusable Procedures 11
3.4 Agentic Workflows and Topologies 12

4 Executable Update Targets **13**
4.1 Tools and Code Patches 13
4.2 Repository and Runtime Logic 14
4.3 Everything Is a Plugin 15

5 Theory of Harness Self-Evolution **16**
5.1 An Evolutionary Perspective on Harness Search 16
5.2 Generalization and Retention from a Continual PAC-Learning Perspective 17

6 Evaluating Harness Self-Evolution **21**
6.1 Four Claims Behind a Task Score 21
6.2 Current Evaluation Practices and Benchmarks 22
6.3 Longitudinal and Dynamic Evaluation 31

7 Applications of Harness Self-Evolution **34**
7.1 Personal Assistance and Developer Workspaces 34
7.2 Enterprise Automation 35
7.3 Scientific Discovery 36
7.4 Robotics and Embodied Interaction 36
7.5 Public Services and Clinical Systems 37

8 Societal and Economic Implications **37**
8.1 Productivity, Work, and Education 38
8.2 Economic Distribution and Market Structure 39
8.3 Innovation and Public-Service Capacity 40
8.4 Privacy, Security, and Systemic Stability 40
8.5 Governance and Institutional Power 41

9 Future Directions **42**
9.1 Model–Harness Co-Evolution 42
9.2 Optimization Algorithms for Heterogeneous Harness Search 42
9.3 Infrastructure for Evolving State 43
9.4 Dynamic Evaluation and Benchmark Refresh 43
9.5 Safety Boundaries for Self-Modifying Systems 43
9.6 Governance, Provenance, and Human Oversight 44

10 Conclusion **44**

1 Introduction

Progress toward artificial general intelligence (AGI) requires agents whose competence is not fixed at deployment. An agent in an open environment meets problems its designers did not anticipate and a task distribution that drifts; a fixed model can generalize to new problems, but a static harness has no mechanism for turning execution feedback into persistent revisions of its instructions, memory, or control logic. Self-evolution relaxes this limit by letting execution experience reshape the system that produces future behavior, so that what an agent learns in one run carries into later ones. The idea connects early work on recursive self-improvement and self-referential problem solving with current self-evolving agents [1–4]. An update, however, is not an improvement: for a system that keeps operating after deployment, each accepted change has to be shown to help rather than assumed to, and because the change persists, so does any fault it introduces (Section 8).

Self-evolution can target either the base model or its execution harness. Model updates alter internal weights; the harness is the external runtime that turns a general-purpose model into an operating agent by orchestrating prompts, memory, workflows, and executable tools across runs. We focus on harness self-evolution for three reasons. First, the harness largely dictates deployed behavior: under a fixed model, performance varies substantially across scaffolds, allowing systems to capture task constraints, failures, and reusable procedures without the cost of retraining [5–7]. Second, harness evolution requires no access to model weights, so it applies to black-box, proprietary, or API-hosted models whose weights cannot be changed, and it remains open under the deployment and cost constraints of Sections 7 and 8. Third, harness artifacts expose a readable and locally editable surface: instructions, memory entries, and code changes can be inspected, versioned, and compared. Model checkpoints can also be versioned, but they do not ordinarily expose a behavioral revision in this form.

We therefore hold the base model fixed throughout. This is a choice of scope, and what it settles is narrow: a comparison that enforces this control excludes model updates as a source of the reported difference. It does not make a harness gain easy to establish. Four questions remain, and Section 6 takes them up: whether the reported number was measured or picked out of a search; whether the accepted change operated in the scored runs; whether the difference came from it rather than from run-to-run noise, extra inference compute, or a changed evaluator; and how far the difference extends beyond the tasks that produced it.

Recent work on harness self-evolution spans memory maintenance, prompt and workflow optimization, tool library synthesis, reflective mechanisms, autonomous evaluation, and multi-agent coordination [6–10]; Figure 2 traces its growth and classifies a representative subset by primary emphasis. These works characterize "evolution" in different terms: some by the mutated artifacts, some by the driving feedback signals, others by the search and optimization algorithms, and many only by terminal benchmark scores, so terminology and experimental standards do not line up across papers. The field therefore needs a shared unit of analysis. We organize the literature by the *update target*, what a proposal modifies, because it identifies the state that later runs inherit; feedback source, mutation granularity, admission checks, inference cost, and safety constraints then enter as properties of an update that affect what its evaluation can show, not as competing ways to classify it.

We formalize harness self-evolution under a fixed base model as an operational feedback loop subject to two conditions: the system's own execution feedback drives the update through a mechanism that stays in place afterwards, and the accepted state is persisted and actually loaded by a later run (Section 2). These conditions describe the lifecycle we focus on, not a filter on what we review: the offline optimizers whose mechanisms these methods reuse, and systems whose changes never persist, are discussed alongside methods that run the full loop. The taxonomy separates artifacts that fixed host components interpret (instructions, memory, and workflow

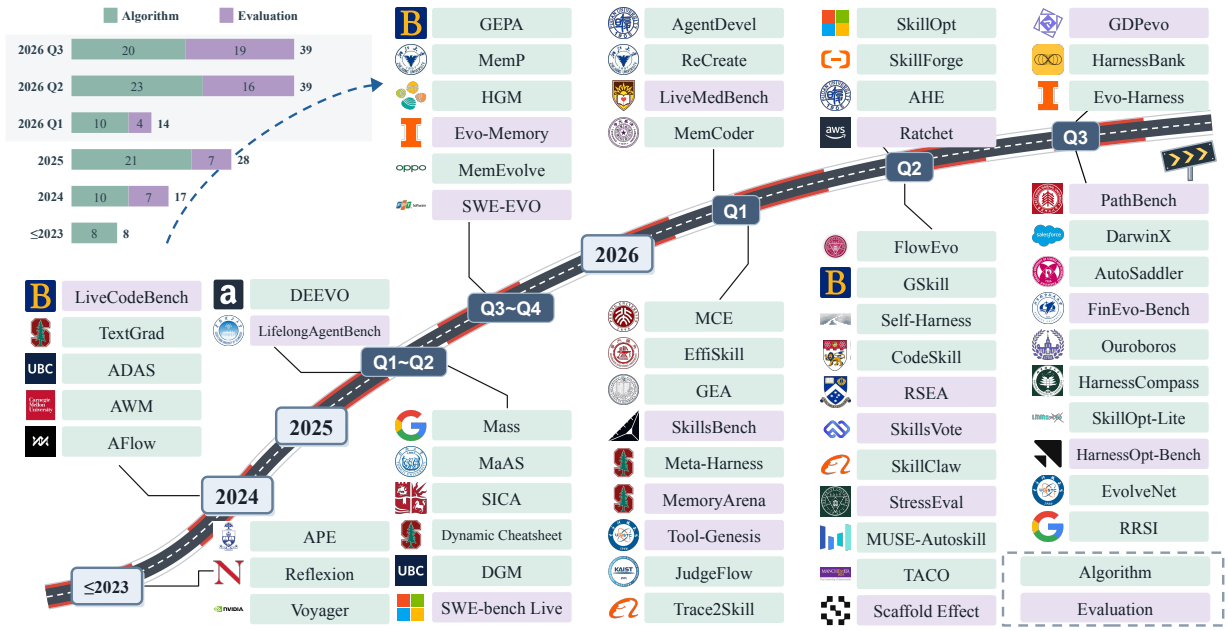


Figure 2 Growth and classification of harness self-evolution work. The inset counts each work drawn in Figure 3 once, at the period of its first public version, split by primary emphasis into update algorithms and evaluation, for works released through September 2026; every date comes from a public record except the one product page dated from the authors' information, and the ledger in the repository names it. The road places representative systems along the same periods, tracing the field's progression from early prompt, memory, and workflow systems to executable components and runtime logic; the systems it names are examples rather than the roster the inset counts. The classification is descriptive, and inclusion does not imply continuing self-evolution or measured improvement.

controls) from code the host runs directly, which ranges from callable tools and harness source to plugin compositions (Sections 3–4).

The four questions above become four claims a reported gain can rest on: *selection*, that the number was measured rather than picked; *activation*, that the accepted state operated in the scored runs; *attribution*, that the difference came from that state and not from one of the other causes above; and *improvement*, that the difference holds away from the tasks that produced it. We operationalize these claims as separate evidence criteria and read each reviewed method's primary comparison against them. The weakest evidence concerns the scored runs themselves: reports seldom record whether an accepted update ran there, or separate its effect from execution variation and added inference compute (Section 6). Section 5 adds two theoretical views: evolutionary search over harness state and a continual PAC-style account of generalization and retention.

Existing surveys organize self-evolution by operational component [3], optimizer feedback [4], externalized infrastructure [11], the role of code [12], or update target paired with signal origin or loop closure [13, 14]. Table 1 compares their harness-relevant coverage. Recent surveys also address evaluation directly. Ren et al. [13] distinguish model-side from scaffold-side updates and call for separate test data, independent final judges, and component substitution. Zhou et al.'s coding-agent survey [15] distinguishes proposal eligibility, runtime activation, and candidate admission, and proposes stage-bypass comparisons under a common budget with actual cost and branch activation recorded. Our contribution builds on these concerns: we operationalize them as comparison-level evidence criteria, apply them to the primary results of reviewed reports, and distinguish an admission certificate from an independent final evaluation and from evidence of retention across later updates. We also analyze how benchmark refresh changes the comparisons needed to establish a harness gain.

This article makes three contributions:

Table 1 How recent surveys cover the harness self-evolution lifecycle. Each cell records the strongest treatment found in the survey: ✓ a dedicated section or subsection that reviews multiple methods, ○ a substantive discussion without a dedicated treatment (including future work), and — no body-text treatment beyond a table entry, passing clause, or explicit scope exclusion. For *base model held fixed*, ○ denotes a definitional or scope-based separation; ✓ would require an experimental control, which none of these surveys supplies. For the four *evaluation claims*, ✓ requires an explicit evidentiary criterion rather than inference from a terminal task score. For *theory*, ✓ requires formal results developed by the survey itself; *societal impact* excludes technical safety. Columns follow preprint order. The Zhou et al. columns are the externalization review [11] and the coding-agent survey [15], whose declared scope is software engineering alone, so its ✓ for applications covers deployed products in that one domain.

Dimension	Gao [3]	Fang [4]	Zhou [11]	Ning [12]	Chen [14]	Ren [13]	Zhou [15]	Ours
Scope								
Base model held fixed	—	—	○	—	—	○	○	○
Update target								
Instructions and prompts	✓	✓	○	—	—	✓	✓	✓
Context and memory	✓	✓	✓	✓	✓	✓	✓	✓
Workflow and control	✓	✓	○	✓	○	○	✓	✓
Executable artifacts	✓	○	—	○	○	✓	✓	✓
Evaluation claim								
Selection	—	—	—	○	—	✓	○	✓
Activation	—	—	—	—	—	—	✓	✓
Attribution	○	—	○	○	✓	✓	✓	✓
Improvement	✓	○	○	○	○	✓	○	✓
Analysis and implications								
Theory	○	○	—	○	—	○	—	✓
Applications	✓	✓	○	✓	○	✓	✓	✓
Societal impact	—	○	○	—	○	—	—	✓

✓ dedicated ○ partial — absent

- **Scope and Operational Taxonomy (§2–§4):** We define harness self-evolution as a persistent operational feedback loop and hold the base model fixed when evaluating harness changes. We then organize the design space by update target, separating modifications to artifacts the model interprets from modifications to executable code the host invokes, and review offline antecedents alongside the methods that reuse their mechanisms.
- **Synthesis and Empirical Review (§3–§4, §6–§7):** Using this taxonomy, we compare existing methods and find that mutation scope and persistence vary independently. Because persisting an update does not show that it helped, we separate the four claims a single task score conflates (*selection*, *activation*, *attribution*, and *improvement*), read the reviewed methods' reports against each, and state the controls needed when the harness and the benchmark drift together. How much checking an update needs depends on where the loop runs: across deployment settings, the admission check, more than proposal generation, has to scale with the cost of a wrong update.
- **Theoretical Perspectives (§5):** We map harness search onto evolutionary paradigms and analyze generalization and retention from a continual PAC-learning perspective. The analysis gives a sufficient condition under which an admission comparison certifies improvement on a round's task mixture, and shows that repeated executions of one task reduce execution noise but add no independent tasks.

Organization. Section 2 formalizes the operational lifecycle of harness self-evolution. Sections 3–4 classify interpreted and executable update targets. Section 5 develops the evolutionary and

continual PAC-style theoretical perspectives. Section 6 examines empirical evaluation practices and dynamic benchmarking. Sections 7–9 cover deployment regimes, societal implications, and open research frontiers, and Section 10 concludes.

2 Definition of Harness Self-Evolution

Table 2 The agent harness by component. *Runtime responsibility* states what the component must supply while a run is in progress; *mechanisms* lists the sub-functions through which it supplies it. A given implementation may fuse several components into one module or omit them entirely.

Component	Runtime responsibility	Mechanisms
Agent loop and execution control	Advances a run from one model call to the next and decides when it ends	Model invocation; tool execution; continuation and stopping rules; cancellation; error recovery
Tool and skill orchestration	Exposes external capability and reusable procedure to the model	Tool interfaces; registration and invocation; skill loading and invocation
Context construction	Assembles what the model sees on each call	System instructions; tool descriptions; conversation history; injected context
Memory and state management	Carries state beyond the current call or session	Reusable memory and retrieval; session persistence; resumption; forking
Context compaction	Keeps the assembled context within budget as a run grows	History summarization; retention and triggering policies; tool-output pruning
Scheduling and coordination	Distributes work beyond a single loop	Sub-agent delegation; task scheduling; background execution
Execution infrastructure	Bounds what a run may do and records what it did	Sandboxing; permissions and approval policies; storage; model adapters; logging and replay

Harness. Following DeepSeek Harness's view of an agent as a model combined with a harness [16], we define a harness H as the software and configuration surrounding a model M that govern its execution and interaction with a task environment E. Table 2 summarizes its functional components, including the agent loop, tool orchestration, memory, skills, and context compaction. These are composable capabilities rather than mandatory components of every harness. Model adapters belong to this surrounding software, while model weights belong to M. A rollout τ, comprising the interaction trajectory and its outcome, is generated as

$$\tau \sim \mathrm{Run}(M, H, E). \tag{1}$$

A rollout is scored by an evaluation procedure J, comprising the task scorer, any verifier used to decide the outcome, and the bookkeeping that records it, giving $y = J(\tau)$. We place J outside H by stipulation, and Sections 5 and 6 keep it outside the surface a candidate may modify, so that a change in J is a change in the measurement rather than a harness update. Internal critics and provisional verifiers may belong to the mutable harness, provided an external J judges the final outcome. A system that exposes this final scoring or verification procedure to its own proposals breaks that stipulation: the code is then part of what the update may rewrite, and its scores before and after an update no longer measure the same quantity. Section 6.2.1 treats this failure separately from the statistical failures of selection, because the remedy is structural rather than statistical.

Harness self-evolution. We define harness self-evolution as an iterative optimization process in which an agent system uses batches of execution experience to revise its own harness, with the aim of improving task performance or the resources a task consumes, while keeping the backbone

model M fixed. The resource half of that aim is not a separate claim: Section 6.2.3 reads a score together with the inference budget that produced it. At update round k, the current harness H_k interacts with a batch of sampled task environments to produce

$$\mathcal{B}_k = \{\tau_{k,i}\}_{i=1}^{b}, \qquad \tau_{k,i} \sim \mathrm{Run}(M, H_k, E_{k,i}), \tag{2}$$

where b is the batch size, $E_{k,i}$ is a sampled task environment, and each rollout includes its interaction trajectory, outcome, and available feedback. The harness is then updated as

$$H_{k+1} = \mathrm{Update}(H_k, \mathcal{B}_k). \tag{3}$$

Here, k indexes update rounds. As in minibatch training, different rounds may use different task samples or revisit samples from a fixed training set; temporal task arrivals and changing task distributions are not required. Equation (3) abstracts diagnosis, candidate generation, evaluation, and retention. The update procedure may internally maintain historical feedback and candidate archives or collect additional evaluation rollouts. Rejected revisions leave $H_{k+1} = H_k$, and single-rollout updates are recovered by setting $b = 1$.

Two indices run through the article and should not be conflated: k counts update rounds, and i counts rollouts within a round. The b rollouts of Equation (2) need not come from b distinct task environments, which is why Section 5 counts tasks rather than rollouts when it asks what a comparison estimates. That section also refines Equation (3): what Update keeps internal here, the feedback history and the candidate archive, becomes an explicit state there, from which H_k is recovered by a deployment step.

Recursion refers to the feedback between harness revision and experience generation: adopted revisions govern subsequent execution, and the resulting rollouts inform further revisions. We designate an update process as harness self-evolution when two conditions hold.

1. **Operational feedback and update continuity.** Within the operating lifecycle of the agent, feedback from its own executions is used by an algorithmic process to form or select the revision, and the update mechanism remains in place after the transition, so later executions can trigger further updates. External environments and automated evaluators may supply evidence, and humans may give feedback or control admission, but a human does not simply author the payload and install it as the next state. This excludes manual configuration, and design-time optimization that terminates after exporting a frozen artifact: an offline optimizer removed once it has delivered a static prompt, workflow, or program leaves a harness behind but no loop, and Sections 3 and 4 mark such systems as offline antecedents.

2. **Persistence across runs.** When a candidate is accepted, the resulting harness state is persisted and actually loaded by a subsequent run, so that it can affect rollouts it did not generate. This excludes candidates evaluated and discarded inside a search, within-run reflection that never changes later execution, and passive logs that are stored but never read by the harness. Whether the loaded state then operated in those runs is the activation claim of Section 6.1, not a condition of scope.

These conditions define a repeatable capability rather than a minimum number of observed rounds. An experiment may contain a single operational update and still demonstrate the mechanism, provided the accepted state stays inside the same feedback and update lifecycle. Ordinary execution under unchanged rules, such as appending a session log or triggering a fixed compaction policy, meets neither condition, because nothing was revised; revisions may instead change those rules, or distill reusable knowledge and skills that guide later execution. Methods that revise the update mechanism itself add a further level of recursion, and Section 4.1 covers the cases that change a memory-management process rather than its contents. Improvement is the objective of the process, not a guarantee assumed for every update: sampling independence, generalization, and

retention require separate analysis.

Naming. This process is called harness self-evolution in the title of this article and harness RSI, for recursive self-improvement, in much of the literature. We retain harness RSI as a conventional short name for this research area, not as a claim that every accepted revision improves performance. We use *self-evolution* for the update mechanism whether or not the accepted state performs better, and reserve *self-improvement* for a beneficial change supported by an explicit comparison. Persistence therefore shows that an update was carried into later runs, not that it operated there or that it helped. A self-evolved state may overfit its feedback, reduce reliability, or introduce safety risks, and Section 6 treats those outcomes separately: each accepted update is a state transition, while an improvement claim rests on comparisons among the parent, the candidate, and later runs. Transient within-run reflection, retry, and test-time computation may contribute to forming a proposal, but whether they amount to harness self-evolution depends on whether they take part in the operational feedback loop and change the state that later runs load. Scope is set by the harness side of an update rather than by what happens to the weights. A system belongs here when it revises its harness from its own execution feedback under the two conditions above, whether or not it also trains the model: co-evolution enters through its harness half, and a report that trains on trajectories its harness produced is read here for what its harness comparison shows. Systems whose only update is to the weights, and fixed components that nothing revises, are outside the lifecycle the two conditions describe, and Sections 3 and 4 list them only where they mark what that boundary separates.

Sections 3 and 4 also review the offline antecedents of these mechanisms—prompt, workflow, and program search that export a frozen artifact—together with boundary cases such as tool synthesis whose output never persists. The prose identifies these lifecycle differences when introducing each work.

Figure 3 maps the literature reviewed under this definition onto three of the sections that follow. Its branches separate what an update may change—interpreted payloads in Section 3 and executable implementations in Section 4—from what an evaluation may claim in Section 6. How the search over candidates is organized is taken up separately in Section 5. The components of Table 2 are the hosts of that first split: Section 3 covers the payloads they interpret, and Section 4 covers the case where a component is itself what the update rewrites. Applications and governance lie outside this map and are treated in Sections 7 to 9.

3 Interpreted Update Targets

This section examines interpreted update targets—instructions, prompt templates, memory entries, and workflow topologies—whose effects are mediated by a fixed model and host. These updates change model-readable content or coordination state while leaving native tools, operator implementations, system-call interfaces, and low-level execution machinery unchanged. Section 4 turns to executable targets that modify those implementations directly. Figure 4 shows both families together with the update loop they sit in.

The historical subsection extracts the propose–evaluate–retain structure that connects earlier self-improvement accounts to modern harness adaptation. The next subsection examines instructions and prompts as optimization targets, followed by persistent memory and reusable procedures. The final subsection considers workflow specifications that change multi-step coordination while leaving the executor fixed.

3.1 From Recursive Self-Improvement to Harness Adaptation

The update loop has precedents in earlier accounts of machine self-improvement, and the three best known differ in which part of it they supply. Good framed recursive improvement as a machine

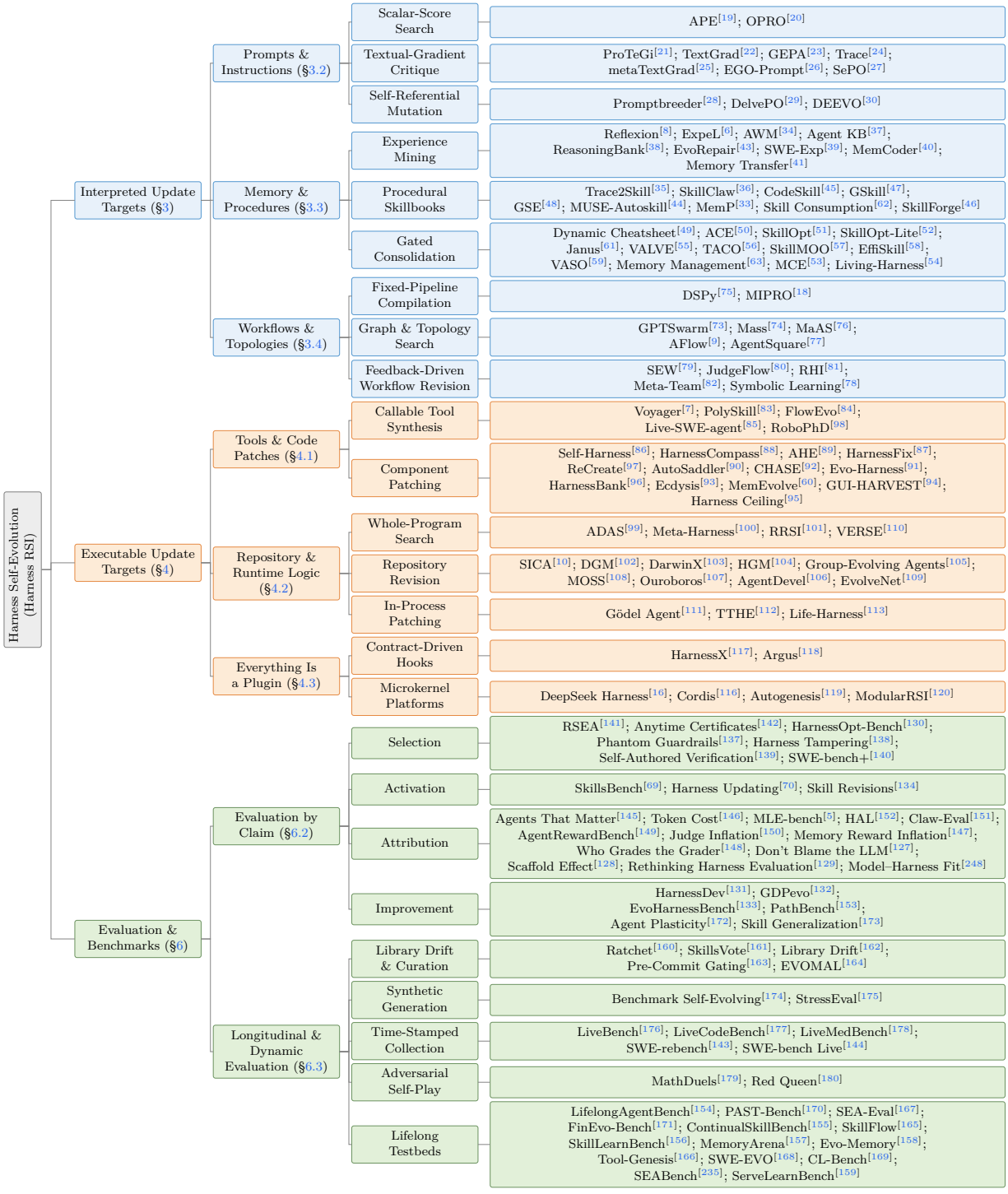


Figure 3 Taxonomy and citation map of harness self-evolution (§3, §4 and §6). Branches correspond to sections, second-level nodes summarize subsections, and third-level nodes identify update mechanisms, evaluation claims, or evaluation settings. A work is listed once, under what it contributes, so the systems §6 uses as examples appear under the update target they change rather than under evaluation. Deployed products are cited in §7 instead, because they persist state across sessions without reporting the feedback-driven update loop of §2; DeepSeek Harness is the one product listed here, beside the Cordis kernel paper, as the plugin platform of §4.3.

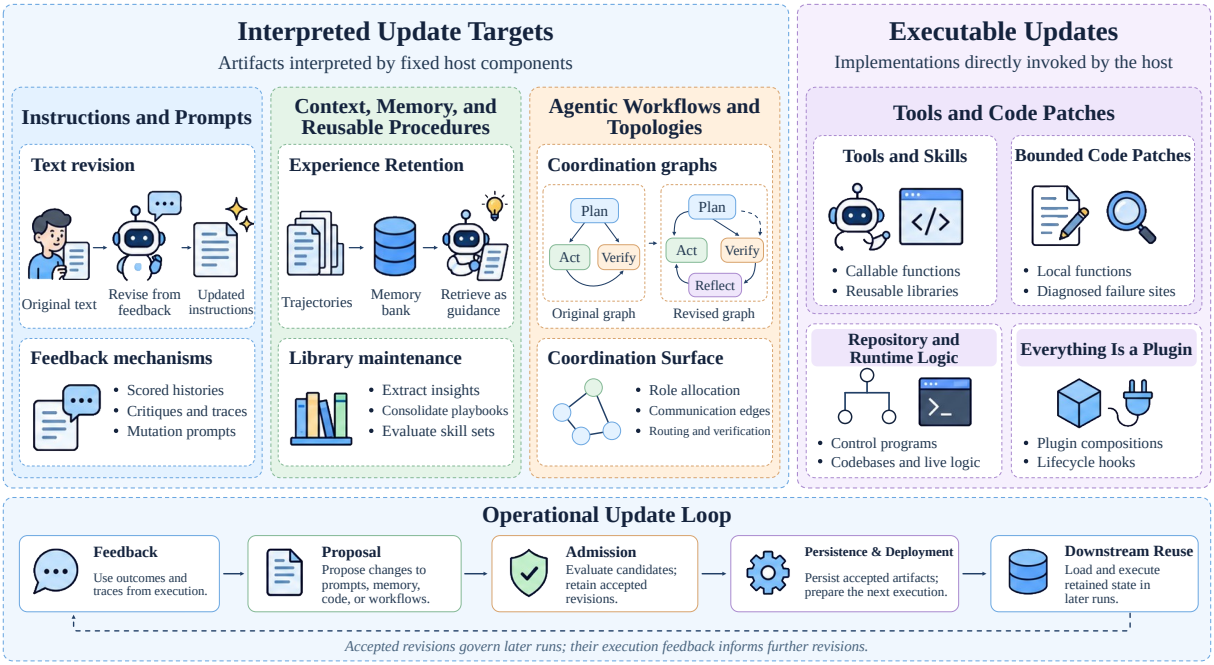


Figure 4 Harness update targets and the operational update loop. Under a fixed base model, interpreted targets comprise instructions and prompts, persistent memory and reusable procedures, and workflow specifications enacted by fixed host components. Executable targets are organized into three parallel software boundaries: tools and code patches, repository and runtime logic, and plugin implementations and compositions. Callable tools and bounded patches share a fixed surrounding executor. The lower panel connects execution feedback, proposal, admission, persistence and deployment, and downstream reuse. Accepted revisions govern later executions, whose rollouts inform further revisions. Persistence and deployment alone establish neither operational activation nor improvement.

designing a better successor to itself, which is a proposal mechanism with no account of how a successor is checked [1]. EURISKO provided an early implemented example in which learned heuristics, including heuristics for modifying other heuristics, shaped later search: proposal and retention, with evaluation left to empirical performance in the domain [17]. The Gödel Machine made formal admission explicit: a self-rewrite is executed only after the system finds a proof that the rewrite improves expected utility [2]. This provides a conceptual contrast with current empirical admission gates. Section 5.2 studies a narrower statistical certificate: an acceptance rule that controls error over the comparisons it makes under explicit sampling assumptions.

Modern harness adaptation narrows the mutable object from the whole reasoning system to external state around a fixed model. Execution traces, tests, verifiers, or preferences supply empirical feedback for proposing and selecting revisions, and accepted state may then be reused in later runs. Many methods share this propose–evaluate–retain structure while using empirical admission in place of a formal proof of utility improvement.

3.2 Instructions and Prompts

The first target is text supplied directly to the base model: system instructions, prompt templates, few-shot demonstrations, and mutation specifications. Updating these artifacts changes the context that steers token generation, not the native tools, scheduling policy, or host runtime [18]. The effect is therefore behavioral rather than enforceable: an instruction can shift output tendencies, but it cannot by itself impose a runtime invariant, rigid schema, or execution boundary.

Prompt search can be grouped by three forms of feedback. Black-box methods use only candidate-level scores: APE samples instructions from input–output examples, whereas OPRO conditions an optimizer on the scored history of prior candidates [19, 20]. Trace-guided methods

use richer execution evidence, turning task errors, critiques, or full trajectories into proposed edits; this pattern spans direct prompt repair, complementary prompt-set evolution, and graph-structured trace analysis [21–24]. It also extends beyond the task prompt to optimizer prompts, domain-specific deployments, and long-lived system instructions [25–27]. Self-referential methods then make the mutation process itself part of the search: Promptbreeder co-evolves task and mutation prompts, while later variants guide mutation directions or compare candidates through tournaments [28–30]. Across these forms, textual feedback guides candidate generation rather than differentiating through model weights; the accepted artifact remains text whose effect depends on the fixed model following it. Execution feedback thus changes the candidate prompt used in the next round, after which most methods freeze the selected prompt for deployment.

Most prompt optimizers remain offline antecedents: they select a high-scoring prompt and terminate the update loop. Extending prompt updates across an operational lifecycle creates a context-management problem. Accumulated instructions consume a finite attention budget, can reduce retrieval precision as context grows, and increase inference cost [31, 32]. These limitations motivate curation or external memory, but neither mechanism turns prompt text into enforceable runtime control.

3.3 Context, Memory, and Reusable Procedures

The second target is persistent, model-readable state constructed from prior experience and retrieved into later contexts. It includes episodic reflections, cross-task strategies, execution playbooks, and reusable skill files [11, 33]. By storing guidance outside the immediate context window, these mechanisms allow later runs to reuse experience without changing the base model [11]. The update loop converts execution traces into candidate entries or procedures, curates the resulting store, and retrieves accepted artifacts in later runs.

Early systems differ mainly in how long the extracted experience is meant to remain useful. Reflexion stores natural-language feedback for another attempt at the same task [8], making it a within-task boundary case under Section 2. ExpeL and Agent Workflow Memory extend reuse across tasks by extracting general insights or recurring sub-goal procedures from trajectories [6, 34]. Trace2Skill and SkillClaw make the reusable unit more explicit by distilling experience into modular skill documents [35, 36]. Across these systems, the mutable artifact ranges from an episodic note to a persistent library of procedures.

As these libraries broaden, systems vary in both the content they preserve and the abstraction at which they preserve it. Some retain cross-domain experience or reasoning strategies [37, 38]; others capture software repair outcomes and repository structure [39, 40], or extract behavioral rules and study their transfer [41]. Human-maintained rule files provide a boundary case: engineers can turn accepted review comments into rules loaded by later sessions [42], but this does not meet the algorithmic-update condition in Section 2. EvoRepair sits here rather than with executable targets: it accumulates, scores, and reuses vulnerability-repair experiences across long-horizon repairs while the executor stays fixed [43]. Methods that turn traces into procedures likewise differ in how skills are induced and evaluated [44–46], and in the task or domain scope for which they are intended [33, 47, 48].

Persistence also makes library maintenance part of the learning problem. Dynamic Cheatsheet and Agentic Context Engineering (ACE) rewrite and consolidate accumulated strategies, while SkillOpt, SkillOpt-Lite, and Meta Context Engineering (MCE) evaluate revised skill sets or archived candidates before replacing the active library [49–54]; VALVE makes the gate itself statistical, admitting a revision only when a paired comparison on a fixed validation panel clears a margin, and reports the retention it buys over a thousand-task stream [55].

Other systems optimize compression or competing objectives [56–58], constrain updates through specifications [59], or hand the memory-management process itself to the update loop: MemEvolve's outer loop rewrites the code that encodes, stores, retrieves, and manages memories, which we

classify with the executable targets of Section 4.1 [60]. Janus separates the decision from the updater: it wraps an existing update rule and, when a candidate deviates from the recent update direction, compares the previous and candidate memories on a compact set of coverage, boundary, and fresh tasks before deploying either [61]. Measurement work complements these methods by testing whether generated skills are used downstream and which management choices affect task outcomes [62, 63].

Context, memory, and reusable procedures have also entered deployed coding agents, where project memory and file-based skills can persist across sessions [64–67]. Operational deployment, however, does not make that experience executable or reliable: the stored artifacts remain data interpreted probabilistically by the model [68]. A useful skill may not be retrieved and may still be ignored after retrieval [69, 70]. Persistent stores can also propagate stale strategies or prompt-injection payloads across sessions [71, 72]. Memory evolution therefore trades reusable, consolidated guidance against the accumulation of conflicting or unvetted context [61].

3.4 Agentic Workflows and Topologies

Workflow and topology targets specify how fixed model and tool components coordinate across multiple steps: role allocation, communication graphs, conditional routing, and verification or termination cycles [73, 74]. We restrict this category to coordination specifications enacted by a fixed orchestration framework. These specifications may be textual, graph-structured, or serialized in code, but the mutable state determines how tasks are decomposed, intermediate outputs are passed, and execution is routed or stopped; node implementations, tool interfaces, and the orchestration framework remain fixed.

The first axis is the mutation surface exposed within the workflow. At the narrow end, DSPy optimizes prompts and demonstrations inside a programmer-defined pipeline while leaving control flow fixed, and MIPRO extends that compilation to a joint search over instructions and demonstrations across the pipeline's stages [18, 75]. GPTSwarm makes connections between fixed operations learnable, Mass jointly searches agent prompts and communication topologies over predefined multi-agent blocks, and MaAS samples query-dependent architectures from predefined operators [73, 74, 76]. AFlow serializes workflows as code that composes reusable operators, then searches the resulting structure and prompts [9]. The generated code does execute, but we classify it here because the update rearranges fixed operators rather than revising their implementations. The distinction concerns the mutation boundary, not whether the representation is code.

A boundary group expands this surface beyond coordination state. AgentSquare searches module compositions while generating new module implementations, and Symbolic Learning jointly revises pipeline structure, prompts, and tools [77, 78]. We classify them here by their primary contribution to workflow optimization, while Section 4.1 discusses their executable mutations.

The second axis is the feedback horizon over which a workflow is revised. SEW jointly mutates prompts and topology while comparing alternative workflow representations, whereas JudgeFlow uses execution outcomes and block-level diagnoses to revise sequence, loop, and conditional blocks [79, 80]. RHI applies pairwise feedback to textual roles, instructions, communication contracts, and interaction hops within one task episode [81]. Meta-Team instead carries experience across tasks to update a reusable scaffold of agent prompts, teammate profiles, collaboration notes, and shared coordination rules [82].

These axes separate a broad search space from a persistent update lifecycle. Most workflow methods produce a pipeline or graph for subsequent evaluation, or revise it only within the current optimization episode; Meta-Team more directly retains experience across tasks. Persistent workflow evolution therefore requires continued revision of retained coordination state, not merely topology search over a larger space.

Section synthesis. The interpreted targets reviewed above form a comparatively bounded layer of the harness: they change model-readable content and coordination state while relying on a fixed executor. An executable harness can store, select, and interpret these artifacts, so interpreted targets are components of the harness rather than an alternative to it. This bounded mutation surface also aids evaluation because candidate prompts, memories, and graphs can be compared while runtime and tool interfaces remain fixed. These targets therefore remain central to harness design even when they do not expand its executable capabilities. Section 4 turns to updates that modify the execution substrate itself.

4 Executable Update Targets

Section 3 examined model-readable content and coordination specifications interpreted by fixed host components. This section considers updates to executable implementations and runtime component bindings, including tools, middleware, control programs, repository code, and plugins [12]. We classify an update by the object it changes, not by the file format used to express it.

Section 4.1 covers callable tools and bounded code patches under a fixed executor. Section 4.2 covers changes to the control program or to several components that shape end-to-end execution. Section 4.3 covers implementations and compositions managed through plugin interfaces and lifecycle hooks. These are parallel software boundaries rather than capability levels, and they are the lower family of targets in Figure 4. As in Section 3, the taxonomy lists each work once by its main contribution while the prose records any additional mutable surface.

4.1 Tools and Code Patches

This category fixes the surrounding executor and updates an explicit executable unit: a callable tool, a skill implementation, a function, or a designated component file. The boundary limits where a revision may land and makes testing and rollback more local. It reduces the blast radius of a faulty edit, but it does not prevent errors from propagating through the component's outputs or shared state.

Callable-tool systems turn successful execution into independently invoked code. Synthesized functions may be retained for later retrieval or composition while the scheduler and tool-loading machinery remain fixed [7, 83]. FlowEvo makes the boundary with Section 3.4 explicit: it compiles a successful workflow into a validated callable artifact, stores it in a persistent skill bank, and later either executes it directly or supplies its structured description to workflow generation [84]. The first route is an executable update; the second remains an interpreted use of the same record. Live-SWE-agent provides the contrasting lifecycle: its generated Bash and Python tools are discarded with the task container, so it demonstrates tool synthesis without persistent harness evolution [85].

Bounded patching systems use execution evidence to revise a declared component without opening the surrounding agent loop to revision. Some aggregate recurring failures or trace-level diagnoses into localized code or configuration edits, as Agentic Harness Engineering (AHE) does from runtime observability, while others use placement rules or a fixed component workspace to keep proposed changes within tools, middleware, memory, prompts, or subagent configuration [86–93]. GUI-HARVEST diagnoses from repeated executions of the same task and maps recurring modes to bounded source edits, recording each edit's predicted behavioral effect before the run that tests it [94]. MemEvolve belongs here rather than with the memory contents of Section 3.3: its outer loop rewrites the encode, store, retrieve, and manage implementations behind a memory-provider interface, so the update object is a component implementation under a fixed executor, not a stored entry [60]. A harness comparison can also be the setting rather than the contribution: one study runs the same loop on a long-horizon planning benchmark to separate the failures a harness edit repairs from those left to weight training, and its harness arm is read here [95].

Admission and bookkeeping vary within this common mutation boundary, from regression checks and separate validation tasks to file-level rollback. HarnessBank maintains an explicit record: candidates are held in an archive partitioned by semantic descriptor, each must declare the trace beacon its patched path emits, and the evaluation code itself sits in a kernel candidates cannot modify [96]. Sections 6.2.1 and 6.2.2 take up the kernel and the beacon as evidence requirements. CHASE changes what the gate asks: a Challenger proposes benchmark-protocol transformations intended to destroy the candidate's gain over the parent rather than its raw score, and a patch is admitted only if its gain survives every transformation confirmed into the archive [92].

Several methods cross the interpreted–executable boundary without requiring another category. AgentSquare and Symbolic Learning are classified with workflows in Section 3.4 because their main contributions are modular search and joint symbolic optimization, although both also generate executable module or tool implementations [77, 78]. ReCreate is broader than tool synthesis: it edits role, process, tool, and memory components at textual and code levels, but explicitly excludes execution-environment and system-infrastructure adaptation [97]. We therefore treat its tool implementations as a bounded executable surface rather than placing the entire method under repository logic. RoboPhD presents a narrower two-artifact boundary, co-evolving a deterministic Python database-analysis tool with model-readable SQL instructions; the deployed package no longer depends on its evolutionary infrastructure [98].

4.2 Repository and Runtime Logic

This category makes the end-to-end control program mutable. Unlike a local patch whose enclosing interfaces remain fixed, an update may cross component boundaries and revise context construction, tool dispatch, verification and recovery, the agent loop, or the code that proposes later changes. The substrate may be a single complete program, a multi-file repository, or live process memory. The common property is therefore not repository size, but that the update can change how several harness functions compose during execution.

One mechanism synthesizes a complete control program inside a fixed outer search loop. ADAS uses a fixed meta-agent to generate agent `forward` implementations and retains evaluated candidates in an archive [99]. Meta-Harness gives an external coding agent the source, scores, and traces of prior candidates and asks it to synthesize a complete task-specific harness program [100]. RRSI regularizes the same search with annealed edit budgets, leakage screening, and noise-adjusted acceptance floors, trading gain on the evolution set for out-of-distribution transfer [101]. All three are offline antecedents in the sense of Section 2: their output is an executable harness selected for later deployment, while the deployed harness does not continue updating from new task experience.

A second mechanism edits a versioned harness implementation. SICA makes its multi-file Python codebase—including tools, subagents, and control flow—available to the best archived agent, whereas lineage methods branch repository variants and preserve alternatives for later selection or inheritance: the Darwin Gödel Machine (DGM), DarwinX, and the Huxley-Gödel Machine (HGM) [10, 102–104]. Group-Evolving Agents changes the search organization by allowing selected parents to share code patches, tools, and execution experience before producing framework-level patches [105]. Across these systems, archive membership identifies candidates for future evolution; it does not imply that every stored version becomes the active runtime.

Other systems make promotion into the running version explicit through release-candidate regression, reviewed commits, health-checked container replacement, or validation before a merged program is broadcast as shared state [106–109]. VERSE makes the search loop itself the second mutable program: the optimizer rewrites the executor's harness and its own prompts, tools, hooks, and notes, with execution checks for its draft edits, and both the optimizer and the executor keep their weights [110]. MOSS illustrates promotion into a persistent deployment: it rewrites routing, hooks, state management, and dispatch code, then combines a health check with human

authorization before the container swap. Such mechanisms close more of the proposal–promotion–later-use loop than archive search, although their evidence remains specific to the reported tasks and gates.

Runtime updates need not be written to a repository. Gödel Agent uses monkey patching to replace decision methods, action spaces, and modification routines in live Python memory; the revised methods govern later decision steps but disappear when the process ends [111]. TTHE instead evolves executable harness programs from execution traces and carries the selected program into subsequent inputs, so it is persistent across test-time rounds rather than merely an ephemeral interface change [112].

Life-Harness is a mixed boundary case: it adapts environment contracts and model-readable procedural skills together with action realization and trajectory regulation, then freezes the resulting runtime interface for evaluation [113]. Only the latter executable mechanisms belong to this section; its prompts and skill text remain interpreted targets.

Not every iterative software-development loop modifies the agent harness. Harness-of-Harness keeps the harness–model configuration fixed while planning, coding, and testing progressively revise the task's software artifact [114]. We therefore exclude it from this target category.

4.3 Everything Is a Plugin

Plugin-based harnesses organize executable components behind a fixed microkernel, declared interfaces, and lifecycle hooks [12, 115]. The mutable object may be a plugin implementation or the composition that binds tools, memory, providers, agent loops, and verifiers into the runtime. Unlike a task-level graph of fixed operators, these bindings can determine which implementation supplies a runtime service, such as memory storage or tool dispatch, and how it is mounted or replaced. This organization can expose many harness functions without permitting unrestricted repository edits: the kernel decides which hooks exist, which dependencies are valid, and how a component may be mounted or replaced.

DeepSeek Harness illustrates the execution substrate rather than a reported self-evolution lifecycle: its Cordis kernel connects runtime components through declared dependencies, while append-only records support checkpointing and replay [16, 116]. Feedback-driven plugin systems instead revise component implementations or compositions under typed hooks, dependency constraints, and activation checks [117–120]. HarnessX provides a concrete instance by checking hook types, singleton constraints, and smoke tests before committing an AEGIS-proposed configuration. The contribution of this category is not a larger mutation radius, but a more explicit contract around replacement and activation.

Those contracts support local checking and recovery, not complete safety. A sandbox can restrict a candidate's permissions, and a typed hook can reject an incompatible binding, but neither establishes that the resulting behavior is correct. Within runtime-managed state, checkpoints, dependency reconciliation, and component replacement may restore a prior logical configuration [116]. They cannot by themselves reverse an API request, database mutation, outbound message, or physical action already committed outside that boundary; even a runtime built for rollback leaves a sent message sent [116]. Such effects require prevention, idempotent operations, or compensating actions rather than software rollback.

Section synthesis. The executable targets reviewed above differ in where they bound a revision: at a declared component under a fixed executor, at the control program that composes several harness functions, or at the interfaces a kernel exposes. A wider bound lets an update change more of how the harness executes, but it does not make the update persistent. Within repository and runtime logic alone, ADAS, Meta-Harness, and RRSI select a program offline, MOSS promotes revisions into the running container, and Gödel Agent's live patches disappear with the process. Mutation scope and persistence are therefore separate properties of a method. Across all three executable categories,

operability, execution coverage, admission cost, and recovery remain evaluation requirements, not properties the representation guarantees. Section 5 gives theoretical perspectives on the proposal and admission steps that interpreted and executable targets share, and Section 6 asks what a reported comparison of such states can support.

5 Theory of Harness Self-Evolution

We examine harness RSI from two theoretical perspectives: optimization and generalization. Turing-complete harness languages can express arbitrary computable control logic in principle, although interfaces, permissions, and resource budgets constrain deployed systems. Expressivity alone does not explain how to find a useful program or establish that it helps. Many reviewed methods perform derivative-free search over discrete harness state, using objective evaluations and rollout feedback to guide revisions (Section 5.1). Continual harness evolution then raises questions of generalization and knowledge retention, for which PAC learning and its continual-learning formulations provide useful theoretical foundations (Section 5.2). Recent work makes these limits explicit: Harness Evolution as Learning studies approximation, generalization, and optimization constraints, while Reliable Self-Evolution derives adoption conditions whose ceiling depends on verification and evaluation cost [121, 122].

5.1 An Evolutionary Perspective on Harness Search

Harness self-evolution can be understood through the evolutionary algorithm (EA) framework of representation, variation, evaluation, and selection. A candidate encodes an executable harness configuration, which may include instructions, tool interfaces, control flow, or reusable skills. Execution on a collection of tasks supplies empirical fitness, while an optimizer proposes modifications and decides which candidates to retain. Empirical fitness supports selection, while traces, diagnostics, and natural-language critiques can guide proposal generation. These methods generally do not differentiate task performance through the discrete prompt, workflow, or code edit. The EA framework captures shared search structures without reducing the available feedback to a scalar or asserting that every harness update implements an evolutionary algorithm. For skill evolution, the search can operate on an individual skill or a skill library within the surrounding harness. Table 3 organizes these methods by the evolutionary paradigms and mechanisms they instantiate or resemble. We use the broader EA framework because the surveyed methods include evolution strategies, archive-based program search, and quality-diversity search, beyond conventional genetic algorithms. The categories are not mutually exclusive: a system may combine one selection strategy with several variation and resource-allocation mechanisms.

The simplest correspondence is elitist single-offspring search: a candidate is generated from the incumbent and replaces it when evaluation indicates an improvement. MCE explicitly adopts a history-informed $(1 + 1)$ evolution strategy [53], while SICA, SkillOpt, SkillOpt-Lite, and the skill-refinement path of SkillClaw exhibit related selection structures [10, 36, 51, 52]. These methods differ in how they generate edits and use feedback, but share the principle of retaining a preferred incumbent across iterations. Self-Harness extends this pattern by proposing multiple edits from a common incumbent and evaluating them before promotion [86]. Its search therefore resembles an elitist $(1 + \lambda)$ scheme, although merging several compatible accepted edits differs from selecting a single survivor in canonical $(1 + \lambda)$-ES.

Archive-based methods broaden the search beyond the current incumbent. DGM, DarwinX, and HGM retain candidate programs and organize subsequent exploration through parent selection, modification, and lineage expansion [102–104]. Their correspondence is to evolutionary search over programs; this does not imply the use of classical tree-based genetic programming operators. HarnessBank further structures its archive using semantic descriptors and competition within cells, following a MAP-Elites-style quality-diversity design [96]. This organization preserves

candidates addressing different mechanisms instead of ranking the entire archive solely by one global score. HGM adds a complementary mechanism: Thompson sampling allocates lineage-expansion and candidate-evaluation budgets, coupling evolutionary search with bandit-based resource allocation [104].

Variation also differs from conventional fixed mutation and crossover operators. In these systems, language models can use execution traces and failure diagnoses to propose targeted changes to natural-language instructions or executable code. Recombination combines content from different candidates, as in DarwinX, MCE, and HarnessCompass, or merges compatible edits, as in Self-Harness [53, 86, 88, 103]. The strength of this correspondence depends on the surrounding search procedure. For example, Trace2Skill consolidates trajectory-derived patches, which resembles recombination at the level of information reuse, but patch consolidation alone does not establish a complete evolutionary selection loop [35]. Similarly, HarnessCompass's component-wise optimization and subsequent combination resemble cooperative coevolution, while lacking an explicit implementation of independently evolving subpopulations [88].

Selection need not reduce candidate quality to a single task score. Meta-Harness considers performance–cost trade-offs, while Self-Harness requires non-degradation on both held-in and held-out splits and improvement on at least one [86, 100]. SkillMOO applies the same principle one level down, selecting among skill sets under several competing objectives rather than one [57]. Meta-Harness and SkillMOO consider competing objectives, while Self-Harness applies a dominance-like acceptance condition across evaluation splits. These are correspondences at the selection level, rather than evidence that the systems implement a complete algorithm such as NSGA-II. Together, the mappings in Table 3 distinguish how systems generate variation, preserve alternatives, and allocate evaluation effort. They also expose a shared limitation: evolutionary selection operates on observed fitness, so retaining an empirically better candidate does not by itself establish improvement on unseen tasks or preservation of earlier capabilities. The following subsection examines these generalization and retention requirements.

5.2 Generalization and Retention from a Continual PAC-Learning Perspective

Evolutionary search explains how harnesses achieve higher empirical fitness, but empirical selection alone does not establish improvement on unseen tasks. Successive revisions must also preserve previously acquired capabilities. The continual PAC-learning framework motivates studying these requirements jointly [123]. We use this perspective to distinguish sample stability, historical retention, and reliable candidate selection. The bounds below give sufficient conditions under explicit sampling assumptions; they do not establish PAC learnability of the surveyed systems.

Continual refinement and its scope. Let X_k contain the complete evolution state after round k, including the current harness, skill library, candidate archive, and feedback memory, and let $H_k = \mathrm{Deploy}(X_k)$ denote the resulting executable system. Write $X_k = \mathcal{A}_k(X_{k-1}, D_k; U_k)$, where D_k contains n_k independent tasks from $\mathcal{D}_k$ and U_k represents auxiliary randomness and any external evaluation samples. For the stability argument, task draws are independent within and across rounds, the distributions are exogenous, and auxiliary random sources are independent of these draws. Feedback generated from a task remains part of the evolution process, rather than an independent observation. We hold the backbone model and evaluator fixed and use a bounded task loss $\ell(H, z) \in [0, 1]$, averaged over execution randomness. Define

$$R_s(H) = \mathbb{E}_{z\sim\mathcal{D}_s}\ell(H, z), \qquad \widehat{R}_s(H) = \frac{1}{n_s}\sum_{i=1}^{n_s}\ell(H, z_{s,i}). \tag{4}$$

This formulation covers both skill updates and broader harness changes. Adaptive curricula, such as Voyager's [7], require an extension of the exogenous-sampling model, or a separate evaluation

Table 3 Harness and skill optimization methods mapped onto evolutionary algorithm paradigms. *Mapping rationale and scope* states what each correspondence rests on and where it stops: a method appears in a row when its search exhibits that paradigm's structure, which is weaker than implementing the algorithm. Rows are not mutually exclusive, so a method may appear in several. Section 5.2 takes up the limitation they share, that selection acts on observed fitness.

EA Paradigm / Mechanism	Mapping Rationale and Scope	Representative Methods
Elitist $(1+1)$ search / evolution strategies	A single candidate is compared with the incumbent, retaining the better solution. MCE explicitly adopts a history-informed $(1+1)$-ES; the other methods share its selection structure. For SkillClaw, this mapping applies to refinement of existing skills.	SICA [10]; MCE [53]; SkillOpt [51]; SkillOpt-Lite [52]; SkillClaw [36]
Elitist $(1+\lambda)$-style search	Multiple edits are proposed from the same incumbent and evaluated before acceptance. Merging compatible accepted edits differs from canonical $(1+\lambda)$-ES survivor selection.	Self-Harness [86]
Archive-based evolutionary program search	Programs serve as evolving individuals. Parent selection, code modification, and lineage retention organize the search. This is a program-level EA correspondence, rather than evidence of classic tree-based GP operators.	DGM [102]; DarwinX [103]; HGM [104]
Recombination / crossover	Complementary content from prior candidates is combined. MCE explicitly defines agentic crossover. Trace2Skill provides only a patch-consolidation analogy, without a complete evolutionary selection loop.	DarwinX [103]; HarnessCompass [88]; Self-Harness [86]; MCE [53]; Trace2Skill [35]
Quality-diversity / MAP-Elites	Semantic descriptors partition the archive into cells, with elites retained within each cell. HarnessBank explicitly draws on MAP-Elites, while modifying parent selection and adding verification gates.	HarnessBank [96]
Cooperative coevolution	Component-wise optimization followed by combination resembles cooperative coevolution. The correspondence is limited: independent evolving subpopulations are not explicitly maintained.	HarnessCompass [88]
Pareto selection in multi-objective EAs	Performance–cost frontiers or multi-criterion acceptance gates resemble Pareto-based selection. This is a selection-level correspondence, not an implementation of a complete multi-objective EA such as NSGA-II.	Meta-Harness [100]; Self-Harness [86]; SkillMOO [57]
Bandit-guided evolution	Thompson sampling allocates lineage-expansion and candidate-evaluation budgets, combining evolutionary search with bandit-based resource allocation.	HGM [104]

stream.

Stability and reusable structure. Algorithmic stability relates sensitivity to individual observations to generalization [124]. Let $H_k^{(s,i)}$ be obtained by replacing $z_{s,i}$ with an independent task from $\mathcal{D}_s$ and rerunning the entire downstream evolution process with coupled random sources. In particular, later feedback, archive contents, and selection decisions must be recomputed. Define

$$\beta_{s,k} = \frac{1}{n_s} \sum_{i=1}^{n_s} \mathbb{E}\left[\left|\ell(H_k, z_{s,i}) - \ell(H_k^{(s,i)}, z_{s,i})\right|\right], \qquad s \le k. \tag{5}$$

A replace-one exchangeability argument then gives

$$\left|\mathbb{E}\left[R_s(H_k) - \widehat{R}_s(H_k)\right]\right| \le \beta_{s,k}. \tag{6}$$

This controls the absolute value of the expected signed gap, not the expected absolute gap or a high-probability deviation. Trace2Skill's aggregation of trajectory-local lessons [35], SkillOpt's structured refinement [51], and PolySkill's polymorphic abstraction [83] are consistent with seeking reusable structure. However, abstraction and small edits do not themselves bound $\beta_{s,k}$: a single observation can still change a shared rule or the selected lineage. Establishing stability requires algorithm-specific analysis; task-replacement experiments provide an empirical sensitivity diagnostic, rather than a proof.

Retention requires controlling accumulated regressions. Generalization and retention are distinct. Define $F_{s,k} = R_s(H_k) - R_s(H_s)$, where positive values indicate forgetting and negative values indicate backward transfer. Equation (6) implies the decomposition

$$\mathbb{E}R_s(H_k) \le \mathbb{E}\widehat{R}_s(H_s) + \beta_{s,s} + \mathbb{E}F_{s,k}. \tag{7}$$

This is not a retention guarantee unless forgetting is separately controlled. For example, if each subsequent update satisfies $R_s(H_u) - R_s(H_{u-1}) \le \varepsilon_{s,u}$, then telescoping gives

$$F_{s,k} \le \sum_{u=s+1}^{k} \varepsilon_{s,u}. \tag{8}$$

Thus, even small tolerated regressions can accumulate. Self-Harness uses split-wise non-degradation checks [86], HarnessX tests previously passing tasks and isolates variants [117], and AHE uses execution evidence to identify and roll back harmful changes [89]. These mechanisms address empirical retention, but historical replay alone does not certify the population inequalities in Equation (8). A stronger protocol allocates explicit regression budgets to historical task groups and assesses them with appropriate uncertainty control. It must evaluate the deployed composition: independently successful edits can interact after merging, and variant isolation still depends on the behavior of the shared router.

From empirical selection to confidence-adjusted acceptance. Let $\bar{R}_k(H) = k^{-1}\sum_{s=1}^{k} R_s(H)$ be risk on the uniform historical mixture. Suppose round k constructs at most K candidate harnesses before accessing a fresh validation set V_k of m independent tasks from this mixture. Conditional on the preceding history and the constructed candidates, V_k is independent of candidate construction.

For candidate H', define

$$\begin{aligned}\Delta_k(H') &= \bar{R}_k(H') - \bar{R}_k(H_{k-1}),\\ \widehat{\Delta}_k(H') &= \frac{1}{m}\sum_{z\in V_k}\left[\ell(H',z) - \ell(H_{k-1},z)\right].\end{aligned} \tag{9}$$

Each paired difference lies in $[-1,1]$. Hoeffding's inequality [125], conditional on the history, and a union bound over at most KT comparisons, where T is the number of rounds, imply that, with probability at least $1-\delta$, simultaneously for every evaluated candidate and round,

$$\Delta_k(H') \le \widehat{\Delta}_k(H') + \sqrt{\frac{2\ln(KT/\delta)}{m}}. \tag{10}$$

Accepting a candidate only when the right-hand side is negative therefore certifies improvement on that round's mixture, under these assumptions; otherwise the incumbent can be retained. In the terms of Section 6.1, this is the improvement claim with its three indices set: the reporting tasks are fresh draws from the uniform historical mixture, the baseline is the incumbent H_{k-1}, and the round is the one at which the candidate is accepted. Here the same tasks V_k decide admission and report the gain, which the selection condition rules out for an uncorrected score; the union bound over KT comparisons is what makes this use valid, and reusing V_k without that correction, or in a later round, would not be. This sufficient condition is conservative, but makes explicit why a positive empirical gain alone is insufficient. The guarantee applies to the evaluated candidate, not to an untested merge or a further revision made after inspecting V_k. Nor does it imply improvement on every historical distribution, on an arbitrary future distribution, or at any later round.

HarnessBank provides a concrete step toward uncertainty-aware admission through task-level paired comparisons and a significance gate [96]. However, its screening tasks are sampled from the training set, and its stated per-comparison threshold does not establish the fresh-data and simultaneous error control assumed in Equation (10). SkillOpt and SkillOpt-Lite gate updates using validation gains [51, 52], while MCE stores validation results in its evolutionary archive [53]. Such feedback makes admission adaptive. Similarly, Self-Harness's fixed held-out set influences later candidates through repeated promotion decisions, even though its task contents are hidden from the proposer. Reusing a holdout in this way requires an adaptive-valid analysis or a controlled reuse mechanism [126]; a union bound alone does not restore the missing independence.

Lack of a per-round certificate does not invalidate evaluation on a separate, untouched final test set. HarnessBank and MCE's offline protocol report such a separation, and SkillForge reserves a final temporal partition for evaluation [46, 53, 96]. These protocols provide evidence about the final system under the evaluated test conditions, rather than certifying every accepted update. Temporal tests additionally require care when interpreting results under distribution shift.

What these guarantees measure. The independent unit is a task, not an individual rollout: repeated executions of the same task can reduce execution noise but do not create new independent tasks. Equation (10) also holds for unbiased bounded estimates of each paired loss when task clusters are independent. Correlated user sessions or support tickets require corresponding grouping or a dependence-aware analysis, and non-additive metrics require their own concentration arguments. Finally, validation only assesses the specified loss. Fresh tasks do not prevent exploitation of a misspecified evaluator. If evaluator risk differs from target risk by at most η for both a candidate and its incumbent, the upper bound on their target-risk difference requires an additional 2η term. Estimating or bounding this discrepancy requires external evidence, such as executable checks or independent human review. Overall, reliable evolution needs reusable

updates, historical coverage, and selection-aware evaluation; none alone establishes all three.

6 Evaluating Harness Self-Evolution

Sections 3 and 4 classified methods by the harness component they mutate. Evaluation asks a different question: what does an observed change in task score establish? In those methods evaluation plays two roles. It selects updates inside the optimization loop, ranking and admitting candidates, and it assesses the resulting system in a report. Offline search already has to keep the two roles apart; continued adaptation adds the question of how an accepted update's effect persists across later updates and tasks. Recent studies show that harness design can change measured agent performance even when the underlying model is fixed, and that published comparisons often leave this factor uncontrolled [127–129]. A terminal score alone does not identify what changed or why.

Reading such a score means checking four claims: three about an accepted update and its effect, and a condition on every number they are read off. Section 6.1 defines them and how they relate. Section 6.2 reviews how current reports support each one, and Section 6.3 extends the claims to repeated updates and changing task streams.

6.1 Four Claims Behind a Task Score

A rise in score can mislead in three ways, and the score looks the same in each. The update may never have been used, it may have been used while something else produced the difference, or it may be the cause on the tasks that prompted it and nowhere else. Underneath the three sits a fourth question, about the runs every claim is read off: the runs a report shows may be the ones its search kept, not a sample of the state it accepted.

Four Claims Behind a Task Score

- **Selection:** The reported number is an unselected estimate, or its inference accounts for selection. For an uncorrected estimate, no outcome on the reporting tasks fed the proposal, the ranking, the admission, or the choice of which round and configuration to report.
- **Activation:** The accepted update operated in the runs that produced the number. It is read from execution records, not scores, at three grades: available, reached, and carried.
- **Attribution:** The difference came from the update, not from execution variation, a larger inference envelope, or a changed evaluator. It needs arms that differ only in the update, a controlled envelope, and a margin beyond measured execution variation.
- **Improvement:** The difference holds away from the conditions that produced it. Three indices locate it: distance from the tasks that formed the update, baseline, and round.

Which claims a study needs depends on what it concludes. Crediting a gain to a particular accepted component needs activation and attribution, and saying that the gain holds beyond the tasks that formed it needs improvement. A comparison of one frozen final state against a baseline needs no activation record, because it compares states rather than crediting a component, but it still rests on selection, attribution, and improvement. A study need not establish all four, and many reasonably report one or two; its conclusion should stop where its evidence stops.

The audit in Section 6.2 operationalizes these claims as separate evidence criteria. Activation meets its criterion when direct records show at least one identified accepted update reached or carried in scored runs; such a record may cover a single case or one configuration rather than every run. Improvement is graded only on task distance, with baseline and round recorded alongside. A held-out partition tests generalization and a probe on an earlier distribution tests retention (Section 5.2), but meeting the distance criterion does not by itself show that the effect is reliable

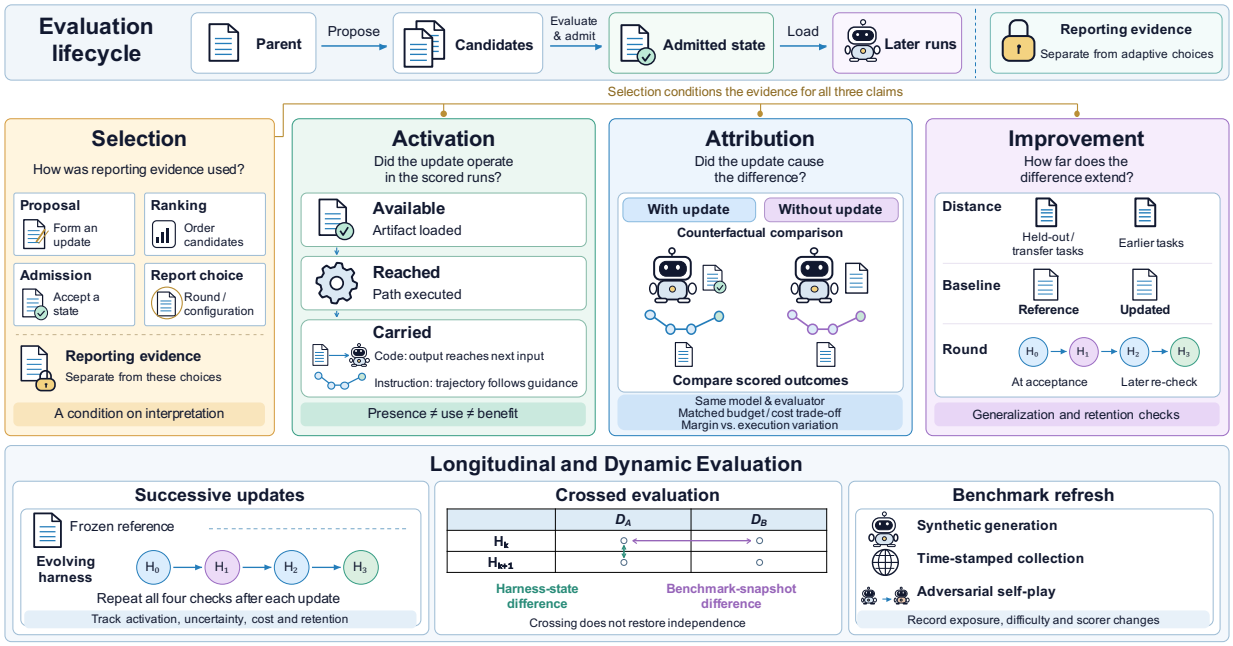


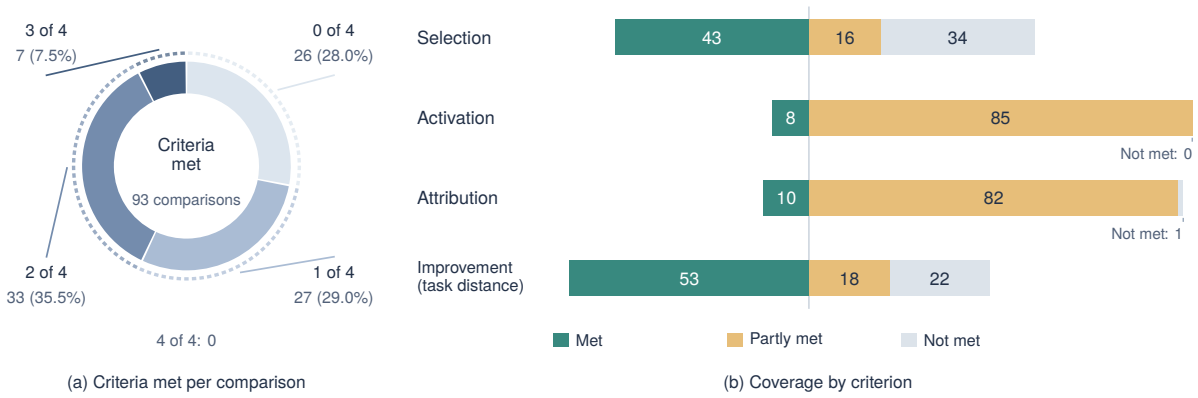
Figure 5 **Four claims behind a reported gain.** Activation, the update operated in the runs that produced the number; attribution, the difference came from it rather than from execution variation, a larger inference envelope, or a changed evaluator; and improvement, the difference holds on tasks away from those that formed it. Selection, at left, conditions the evidence for all three: the number they are read off was measured rather than picked, none of its outcomes having fed the proposal, ranking, admission, or choice of round and configuration. The lower panel carries the claims across rounds. Repeating the checks after each update is what a claim about successive updates needs and a claim about one final state does not (Section 6.3); crossed evaluation and benchmark refresh separate harness change from benchmark change. The controls of Table 6 are what make each checkable.

or that the number was not selected. Selection depends on when evidence was used rather than on what a partition is called. A held-out partition that later ranks candidates, decides admission, or picks the reported round has been spent. A spent partition can still support a claim when the inference corrects for the selection, as the simultaneous comparison bound in Section 5.2 does for admission, but the grading reads the uncorrected numbers that reports give.

Four distinctions carry the reading that follows. Persistence is not activation: a state can be saved and loaded again, which is what puts a system in scope (Section 2), and still leave no mark on the run. Activation is not attribution: whether a component ran is read from execution records, while what it caused is read from compared scores; an ablation that changes the outcome shows that a component operated somewhere without saying in which runs, so we credit activation only on direct execution records. Size is not extent: a difference can come from the update and vanish on a new task set, or hold on a new task set while the credit belongs to a larger inference budget. Distance is not selection: a partition can be held out from the tasks that formed the update and still be spent on ranking, admission, or the choice of round. Beyond these, a valid comparison of one final state shows that state better under the tested conditions, not that each accepted update on the way was sound. Figure 5 sets selection beside the three claims as the condition on the evidence they are all read from, and carries the claims across rounds below them, where a running loop also has to separate harness change from benchmark change.

6.2 Current Evaluation Practices and Benchmarks

Several recent benchmarks build parts of these claims into their protocol. HarnessOpt-Bench keeps its test partition out of reach for the whole search and meters the resources the target agent spends, which covers selection and the inference envelope. HarnessDev and GDPevo score



(a) Criteria met per comparison

(b) Coverage by criterion

Figure 6 **Evidence-criterion coverage across 93 primary comparisons.** One comparison is assessed per method; the two methods without empirical evaluation are excluded. **(a)** Number of criteria met; each outer tick represents one comparison. Zero criteria met does not exclude partial evidence. **(b)** Coverage by criterion; each row totals 93. Activation requires at least one identified scored-run instance, with coverage recorded separately. Improvement grades task distance only. The four-of-four count is not an overall quality score: a final-state comparison need not trace individual component activation (Section 6.1).

evolved harnesses on held-out tasks. GDPevo builds those tasks by recombining business rules spread across its training tasks, to test whether training experience transfers to new combinations. EvoHarnessBench grows the harness an agent is given and measures whether earlier competence is retained [130–133].

A benchmark still supplies only tasks and a scoring routine. Which claim a score supports depends on the comparison it enters and on what its feedback was used for. We therefore read every method in the two update-target branches of Figure 3 against the four definitions, one comparison per method. This *primary comparison* is the result the report leads with: the evolved state against the same-model baseline it is stated against, the pre-update state where the report gives one. Evidence from other experiments in the same report, or from another model, benchmark, round, or comparator, is not carried into its verdicts. The verdicts thus record what the primary comparisons show; they do not rank methods or judge every experiment a report contains. Of the 95 methods in scope, 2 report no empirical evaluation, which leaves 93. Five further works read for the audit fall outside the scope of Section 2 and are not counted. Figure 6 gives the distribution, and Table 5 gives the verdicts.

Corpus and grading. The audit covers the curated literature map in Figure 3, with a cutoff of 8 October 2026; it is not an exhaustive systematic sample of the field. Each verdict grades the evidence the primary source reports for its primary comparison, from the experiments and protocol sections rather than the abstract.[1]

The most common profile in Table 5 meets the selection and task-distance criteria and partly meets the other two. Such a comparison reports a gain on tasks held apart from the formation of the update and from the choices of the search. With attribution only partly met, the margin is not fully separated from execution variation, a larger inference envelope, or a changed evaluator; with activation only partly met, the gain is not tied to the mechanism the update introduced. No comparison meets all four criteria, and six of the seven that meet three miss only activation.

Evidence about the scored runs themselves is the thinnest part of the record: activation and attribution are the criteria least often met (Figure 6), and almost every other comparison meets them partly. For activation, a report shows that the artifact was loaded, or that its path ran

[1] Audit materials: the criteria, the coding procedure and its limits, the rubric with its revision history, and each comparison's verdicts with the passages they rest on.

during search or screening; for attribution, it compares scores without a measured execution variation that the margin clears, or without controlling the inference envelope. The pattern holds across update targets: no prompt optimizer supplies direct scored-run evidence that the revised instruction was followed, although every one reports on held-out tasks, and no repository-level comparison records an identified accepted edit running in a scored run. These are reporting gaps, not evidence that the updates were unused. A separate evaluation study shows what such a record can reveal. Across four agents tested in a sandbox, a rule revision raises the rate of the required action by +0.23 when the skill is preloaded, and keeps about 51% of that action gain on average when the skill body is only available on demand [134].

Where the selection criterion is not met, the failure takes two forms that show up differently in the verdicts. Most often the reporting evidence also fails the distance test: the number is read on the tasks that formed the update, on a partition never shown to be separate from them, or on the round or configuration those tasks selected. Less often a report holds out a partition separate from the tasks forming the update but picks the candidate, configuration, or round on it, and so meets the task-distance criterion but not selection (Section 6.2.1). The split follows the update target. Tools and code patches keep selection clean most often, in 13 of 18 comparisons, and repository-level search least often, in 4 of 16, because whole-program loops often evolve and report on the same benchmark.

Each comparison that meets the task-distance criterion sets one evolved state against its baseline on a held-out or transfer partition; none re-estimates the contribution of a particular accepted update after later revisions. Two reports reviewed here do measure retained performance: VALVE at checkpoint level, with peak-to-final drawdown, and PolySkill by revisiting an earlier task distribution in a secondary case study [55, 83]. Section 6.3.2 describes both and what they leave unmeasured.

The four subsections that follow take the claims one at a time and show what the evidence for each looks like in practice. Five harness-level systems serve as running examples, because their reports describe runtime traces, promotion rules, and evaluation code in enough detail to say where the evidence is thinnest. Table 4 summarizes them.

6.2.1 Evidence for Selection

Selection comes first because the other three claims are read off the runs behind the reported number: if those runs were picked, a margin read off them is biased upward by the search that picked them. One task score can serve four purposes in an update loop—it can generate a candidate, rank candidates, admit one, and be reported—so what a score is evidence of depends on when it was observed and what it was fed back into, and a split named “validation” or “held-out” does not settle that.

The reporting evidence is spent on the proposal when the reporting tasks generate the update. This is the case most specific to self-evolution. A loop that reads the failures of its scored runs and writes a repair from them has used those tasks as a training signal, whatever the split is called and however the candidate is ranked later. SWE-Exp builds its experience bank from search runs on the same 500 SWE-bench Verified issues it then scores [39], and MOSS feeds four tasks into evolution and re-scores the candidate on the same four [108]. No ranking rule or stopping rule repairs this, because the evidence was spent before any candidate existed. Two protocols look similar and spend nothing. AWM scores each WebArena task before its outcome may enter the workflow memory, so no outcome reaches the update before it is scored [34]. Such a number is a valid online measurement of the learning policy, rather than of one frozen final state; what it lacks is distance from the stream, which the task-distance criterion grades. Reflexion adapts inside a task from self-generated tests and never sees the benchmark’s verdict [8]. Its number is also a measurement, but of adaptation within one task, not of an update carried to new tasks.

Table 4 Evidence criteria in five harness-level systems. ✓ met, ○ partly met, — not met in the primary comparison, as in Table 5. Rows are ordered by the number of criteria met. Act. credits identified scored-run cases, not coverage of all runs; Imp. grades task distance only. The last column explains the lowest verdicts.

System / update target	Sel.	Act.	Att.	Imp.	What the report shows	Where it falls short
HarnessBank [96] Component patch	✓	○	○	✓	Evaluation code in a kernel candidates cannot edit; sealed test split; three attempts per task with paired z tests	[Act.] Beacons are logged in screening, not in the sealed-test runs. [Att.] The cost it reports is search budget, not the inference of the scored runs.
Self-Harness [86] Component patch	○	✓	○	○	Promotion rule over held-in and held-out splits; guard visible in traces of the headline run; 22.5% to 52.2% on 180 tasks	[Sel.] The promotion rule consults the held-out split every round. [Att.] Two attempts per task, but no interval or test. [Imp.] The headline pools the held-in half that supplied the edits.
AHE [89] Component patch	—	✓	○	—	Submit-guard and shell-block records in rollouts of the reported round, shown as cases; one evolved layer swapped in at a time	[Sel.] The best round is chosen on the evolution tasks. [Imp.] The headline is scored on those same tasks.
HarnessX [117] Plugin composition	—	✓	○	—	Declared trace feature checked against the next round's traces, shown for one of 15 configurations; per-task inference cost declared	[Sel.] The peak round is reported on the evolution tasks. [Imp.] No held-out tasks.
MOSS [108] Repository logic	—	○	○	—	Health probes on the swapped image; three trials per task	[Sel., Imp.] The same four tasks are the evolution batch and the test set.

The evidence is spent on ranking when the reporting partition also orders the candidates. Offline prompt and workflow search is designed to avoid this: it optimizes on task feedback, ranks on validation, and reports on a partition touched by neither [9, 23]. Once the reporting partition ranks, prunes, or guides candidates, the reported score is the best of the pool on that partition. OPRO shows the instruction with the highest test accuracy, and ProTeGi takes its test figures from the best of a final beam scored on the test split [20, 21].

The evidence is spent on admission when a held-out score enters the promotion rule. This is harder to see in a persistent loop, because a task set can be hidden from the proposer and still shape later candidates through repeated admission decisions. Self-Harness promotes an edit only when its pass count does not fall on either a held-in or a held-out split and rises on at least one, and it never shows the held-out split to the proposer [86]. The held-out pass count is nevertheless half of the promotion rule and is consulted every round, so the split is not independent of the search. A score used for admission is admission evidence, not an independent estimate of the admitted state [135, 136].

The evidence is spent on the choice of what to report when the reported round or configuration is picked on the reporting tasks. HarnessX reports peak accuracy on its evolution tasks and says itself that the numbers carry selection bias and potential overfitting, and its own trajectories show how far a peak can sit above the final state: one configuration peaks at 73.8% and ends its run at 49.5%. AHE carries forward the round with the highest pass rate on the tasks it evolved against [89, 117]. Reporting the maximum of a trajectory, with no stopping rule fixed in advance, turns the reporting metric into an objective of the search, and the same happens when a configuration is chosen instead of a round: SkillOpt's headline row matches the best rows of sensitivity sweeps run on

its reporting benchmarks rather than its stated default, EGO-Prompt reports the best of three repeated runs, and APE reaches human level on all 24 instruction-induction tasks after retuning its hyperparameters and re-running only the five test tasks an earlier version had fallen short on [19, 26, 51].

The fifth failure is different in kind: the candidate changes the record used to judge it. Studies of this failure report patches based on faults that were never observed in execution, direct edits to verification logic, unreliable self-authored checks, and, in SWE-bench, leaked solutions and weak assertions [137–140]. An apparent gain then reflects a changed measurement, not a better harness, and the remedy is structural. HarnessBank keeps its evaluation and bookkeeping code in a kernel that candidates cannot modify, and HarnessX applies the reference test patch before the model patch and blocks writes to test files [96, 117].

The remedies for the four forms of spending are procedural. Protocols reserve reporting evidence from proposal, ranking, and admission, and either specify the reported round and stopping rule in advance or account for their adaptive choice. They treat repeated data-dependent comparisons with anytime-valid procedures [141, 142]. They also collect tasks after a stated cutoff, although a later release date prevents neither adaptive reuse nor shifts in difficulty [143, 144].

6.2.2 Evidence for Activation

A skill can be loaded into context and ignored, and executable code can be installed without its path ever being taken. Evidence that an update operated comes at three grades of increasing strength. None needs a counterfactual run, and each counts only when recorded in the scored runs.

The weakest grade, available, shows that the artifact entered the run. SkillsBench records whether task-provided skills are read or referenced, Voyager measures whether synthesized programs are retrieved, and Lin et al. report a skill-load rate over trajectories [7, 69, 70]. An artifact the agent never acts on also satisfies it, which is why availability alone does not establish use.

The second grade, reached, shows that the changed path executed. Several systems can record it, but most do so outside the scored runs. Symbolic Learning stores each node's inputs, outputs, and tool calls along the executed graph, as the trajectory its training loss is computed from [78]. AutoSaddler records a trace diff before and after each patch in its optimizer's diagnosis sessions [90]. HarnessBank requires each candidate to declare a beacon that fires when the patched path triggers, and rejects during screening any candidate whose beacon never fires [96]. Its beacons are logged on the screening subset, and none is reported for the sealed test behind its headline result. Each of these shows that a path can run; this grade asks whether it ran in the scored runs.

HarnessX records this grade in the runs it reports. Each edit declares the trace feature that must appear if the edit fired, and the declaration is checked against the next round's traces, which on its evolution tasks are the reported runs. On the GAIA / Sonnet 4.6 configuration, whose reported peak is round 11 at 83.5%, a round-10 composite edit declares a call to a named retrieval tool, and the trace analysis behind the reported round finds that the tool fixed retrieval for most newly passing tasks [117]. The record covers one of 15 configurations and shows that the edit fired, not how often across them. HarnessX calls this field an attribution signature. Under our reading it is activation evidence, and attribution (Section 6.2.3) is a further claim. AHE shows, in rollouts of the reported round, the token that marks its submit guard as engaged and the shell-layer refusals that name the protected paths, and Self-Harness pairs its headline run with traces in which the retained guard shapes the trajectory [86, 89]. Both show cases, not rates over the scored runs.

The third grade, carried, shows that the effect of the artifact reached the model or its trajectory. It takes two forms. For a code path, the output has to survive to the model's next input. HarnessX requires a code-bearing candidate to show, before it ships, that its result is still present after prompt assembly, not merely that the call returns [117]. This catches a component that runs correctly and whose result is then dropped in assembly or cut by context truncation; such a

component fires every beacon and changes nothing. For an instruction or a skill, the trajectory has to follow it. Lin et al. score trajectories against a rubric extracted from the skill text and report the resulting adherence rate separately from the load rate, and the two come apart sharply: two agents in their evaluation load skills at almost the same rate, 0.961 and 0.957, while their trajectories follow the loaded skill at 0.350 and 0.757, with adherence for the weaker agent falling from 0.52 at the point of loading to 0.13 at the final turn [70]. Survival is the weaker form and is implied by adherence, since a result the model never sees cannot shape the trajectory. It is still worth recording, because it costs a log line where adherence needs a judge.

The three grades correspond to three rates a report can state next to its task score. For an update u, let $R(u)$ be the scored executions satisfying a declared applicability rule for u, specified independently of whether it fired or succeeded. Within $R(u)$, let $L(u)$ be those that load it, $B(u)$ those in which its declared beacon fires, and $C(u)$ those in which its effect carried. Then

$$\text{load}(u) = \frac{|L(u)|}{|R(u)|}, \qquad \text{reach}(u) = \frac{|B(u)|}{|R(u)|}, \qquad \text{carry}(u) = \frac{|C(u)|}{|L(u)|}. \tag{11}$$

For an instruction or skill, carry is the adherence rate Lin et al. report. For a code path that enters no context, $L(u)$ is read as $B(u)$, so the third rate is always conditioned on the artifact having entered the run: restricting $R(u)$ to executions where the update applies keeps a component written for rare tasks from looking inert, and conditioning the third rate on entry keeps it from being diluted by executions the artifact never entered. A rate is undefined when its denominator is zero; reports should state the corresponding counts rather than replace it with zero. Current instruments report less than this. HarnessBank's gate asks only whether the beacon fires on any task or attempt, so it separates a candidate that never fires from one that fires once, but not one that fires once from one that fires throughout [96]. Lin et al.'s headline rate is per trajectory, which hides partial and time-varying adherence; their instruction-level judgments and phase scores recover part of it [70].

6.2.3 Evidence for Attribution

Three things often move together with a harness update and can produce a difference of the same size: execution variation, the inference envelope, and the evaluator. This subsection takes them in turn and then turns to the counterfactual run, the most direct form of attribution evidence.

Execution variation. Execution variation is estimated by re-running a frozen state on the same tasks; repeated runs estimate noise and do not create new independent tasks. The requirement is to compare the observed gain with the measured variation of the unchanged state, and to state the unit over which that variation is computed, and practice falls short of it in different ways. DarwinX averages five and states the spread on its main benchmark, adding a paired McNemar test on its held-out split, whose $p = 0.45$ it calls suggestive [103]. TACO runs five with matched budgets and reports standard deviations that put its 1.4-point headline margin at about one standard error of the difference, and AutoSaddler reports the mean and standard deviation of three [56, 90]. HarnessBank averages three attempts per task and screens candidates against the spread of the paired differences, not against zero [96]. AHE, Self-Harness, and MOSS run two or three executions per task and report neither intervals nor paired tests [86, 89, 108]. Spread over judges or evaluator seeds does not meet the requirement: RHI averages two judges and three seeds while each iteration executes the agent once, so its spread measures the evaluator, not the execution the update depends on [81]. Neither does spread belonging to a different comparison: ProTeGi repeats its optimization twelve times but reports the resulting standard error against a competing optimizer at its own budget, while the headline sets the evolved prompt against the original one, and the main results average three trials without a spread [21].

A large margin is not a substitute for measured variation. Bounds computed from task counts depend on assumptions about task independence and covariance between arms, while dispersion from another model, split, or configuration need not describe the primary comparison. We therefore credit this condition only when repeated runs, a paired test, or reported raw data distinguish the primary margin from zero under the stated rubric. Self-Harness reports a large AppWorld gain but no interval or test, so Table 4 records partial attribution [86]. Similarly, Harness Evolution Hits a Ceiling reports a held-out gain from 0.164 to 0.297, but the headline pair uses four versus two rollouts on different nights. Dispersion pooled across other configurations does not establish uncertainty for this pair, so its attribution criterion is only partly met [95].

Inference envelope. The inference envelope is the compute spent in the runs being compared. Workflow expansion, retries, critic or verifier calls, extra tools, and larger candidate pools can all raise the final score. A controlled comparison can estimate the total effect of an update, including the resources it induces. The stricter attribution criterion used here asks whether the gain survives control for additional inference effort; it does not treat every total-effect estimate as invalid. A comparison can handle this by matching the envelope, holding the inference budget fixed across arms; by adjusting for cost, reporting performance together with resource use or as a performance–cost frontier; or by declaring the envelope, letting the budget change and reporting the change. Only the first two control it, since a declared change tells the reader that the budget may explain the gain, not that it does not. Current practice covers all three. TACO matches budgets across arms, Meta-Harness and MCE plot accuracy against context tokens as a frontier, and HarnessX declares per-task inference cost with its gains, which falls by a quarter on one benchmark and rises by sixty percent on another [53, 56, 100, 117].

Two budgets are easily confused. Search cost, the rollouts and tokens spent finding the update, belongs to the method; the envelope is the inference compute of the scored runs, and only it can explain a difference between them. HarnessFix reports 57.6M tokens of offline repair cost, and HarnessBank keeps its search within 2.1 times that of its baselines. Neither figure says what the scored runs consumed, and the credited repairs in both add verification or finalization steps inside the agent loop [87, 96]. DarwinX shows the cost of an envelope that is declared and not matched: its matched-model pair runs the unevolved baseline at the provider's default reasoning effort and the evolved harness at high, and on the six tasks it newly solves its own figure shows the evolved harness taking twice the turns and four times the tokens. The reported margin is therefore the joint effect of the new logic and a higher effort setting, and its comparison with Terminus-2 at xhigh effort does not separate the two, because Terminus-2 differs from the base harness in more than its budget [103].

Two envelope changes are specific to self-evolution and easy to miss, because neither appears as an edit to a budget setting: an update that adds retries or verifier calls inside the agent loop raises the number of model calls per task without changing any declared limit, and an update that accumulates skills, instructions, or memory lengthens the context, raising the tokens processed on every later task. The first is compute the unchanged state could also have been given, so it has to be matched or traded off before the gain is credited to the logic; the second is the artifact's own context overhead. This audit records that overhead as part of the update rather than a separate competing cause, while requiring it to be declared. A fixed-total-token comparison can additionally test whether the gain survives a common resource cap. No single cost measure fits every harness, and the declared envelope should cover whichever resources the update changed [145, 146].

Evaluator. The evaluator is the third confound: the score can move because the measurement moved. With reward inflation or a co-evolving grader, a higher score may reflect a change in the stored evidence or in the judge, not in the harness [147, 148]. The corresponding checks are meta-evaluation against expert annotation and detection of judge drift [149, 150]. HarnessBank

reports that on five domains the mechanism its gate credits matches the dominant failure pathology, across domains scored by deterministic verifiers and domains scored by a judge, which argues against a scoring artifact without re-scoring any result under a second evaluator [96].

Counterfactual runs. A counterfactual run changes one component and holds the rest of the run as stable as practical. Component substitution and ablation are the common forms, and they inherit the variation problem above: AHE swaps each evolved layer into its seed alone and reports each swap as a single number on 89 tasks, so the ordering of the layers is not separated from noise [89]. CHASE instead builds the counterfactual into admission, admitting a patch only when its gain over the parent survives every benchmark transformation a challenger has confirmed, so the gain is tested against a change of protocol and not only against noise [92].

Two systems go further and check each shipped edit against the next round's task-level outcomes. HarnessX records, for each edit, the tasks it should unlock, stabilize, or put at risk, and checks them against the following round [117]. AHE does the same and reports how accurate its predictions were: for the tasks an edit would fix, precision and recall are roughly a third and a half, while for the tasks it would break both stay near a tenth, about twice the random baseline [89]. A loop can thus say with some accuracy what an edit helped and can barely say what it harmed. Trace-level checks read the same evidence for a different question, whether a newly passed task used the intended mechanism or exploited the evaluator: HarnessX reports three cases its authors file under reward hacking, including tasks passing by matching format regularities in the verifier, one benchmark's mean reward rising while its exact-match rate stayed fixed, and candidates clearing every gate while their score fell, because the gates check replay, novelty, and structure but not pass rate [117]. All of this evidence is local to the evaluated tasks and configurations; whether the difference extends beyond them is the improvement claim.

6.2.4 Evidence for Improvement

Improvement asks how far a measured difference extends: on which tasks, against which baseline, and at which round. Current evaluations use one of three protocols, and each supports a narrower claim than the definition.

Local failure repair asks whether a candidate can re-solve the task that produced the update. MOSS checks its repairs with execution and recovery evidence on the same four tasks that formed the update batch [108, 151]. Repair shows that the system operates, but it does not meet the task-distance criterion, because the originating task has already shaped the candidate and because re-running only earlier failures benefits from regression to the mean. A stronger repair claim compares the candidate with its unchanged parent under matched repeated executions and includes tasks that previously passed.

Frozen transfer evaluates a selected state without updating it further. AHE transfers a harness evolved on Terminal-Bench 2 to SWE-bench Verified, and Voyager and ExpeL test reuse on new tasks, environments, or domains [6, 7, 89]. A transfer sets the distance index and nothing more, and this one shows how thin that index can be on its own: AHE's frozen harness moves the seed's 75.2% on the 500 SWE-bench Verified issues to 75.6%, a 0.4-point difference the report does not test. Table 4 records AHE's improvement as not supported for a different reason, that its headline result is the in-domain Terminal-Bench 2 number rather than this transfer. Frozen transfer also says nothing about whether the update loop stays reliable over successive revisions, which is the point of the protocols in Section 6.3.

Online trajectories report outcomes while the harness changes along a task stream. ACE varies task order, and SICA evaluates successive code mutations with latency and token use in its utility [10, 50]. A rising trajectory does not show that the difference extends: it may depend on task order, hide regressions on earlier tasks, or reflect growth in context and inference effort, and

Table 5 Evidence-criterion coverage in each method's primary comparison. ✓ met; ○ partly met; — not met, based on the primary source and the criteria in Section 6.1. Each row assesses the comparison the report leads with; evidence from other experiments is not pooled. Act. credits at least one identified scored-run instance, with coverage recorded separately. Imp. grades task distance only, not a reliable positive effect, baseline strength, or persistence across rounds. The literature cutoff is 8 October 2026. Cordis and DeepSeek Harness report no empirical evaluation and are omitted.

Method	Sel.	Act.	Att.	Imp.	Method	Sel.	Act.	Att.	Imp.
ACE [50]	○	○	○	✓	Mass [74]	✓	○	✓	✓
ADAS [99]	○	○	○	✓	MCE [53]	✓	○	○	✓
AFlow [9]	✓	○	○	✓	MemCoder [40]	○	○	○	○
Agent KB [37]	—	○	○	○	MemEvolve [60]	✓	○	○	✓
AgentDevel [106]	○	○	○	✓	Memory Management [63]	✓	✓	○	○
AgentSquare [77]	—	○	○	—	Memory Transfer [41]	—	✓	○	✓
AHE [89]	—	✓	○	—	MemP [33]	○	○	○	✓
APE [19]	—	○	○	✓	Meta-Harness [100]	✓	○	○	✓
Argus [118]	—	○	○	—	Meta-Team [82]	✓	○	○	✓
Autogenesis [119]	✓	○	○	○	metaTextGrad [25]	✓	○	○	✓
AutoSaddler [90]	✓	○	○	✓	MIPRO [18]	✓	○	✓	✓
AWM [34]	✓	○	○	○	ModularRSI [120]	✓	○	○	✓
CHASE [92]	✓	○	○	✓	MOSS [108]	—	○	○	—
CodeSkill [45]	○	○	○	○	MUSE-Autoskill [44]	—	✓	○	—
DarwinX [103]	—	○	○	—	OPRO [20]	—	○	○	✓
DEEVO [30]	○	○	✓	✓	Ouroboros [107]	○	○	○	○
DelvePO [29]	○	○	✓	✓	PolySkill [83]	—	○	○	○
DGM [102]	—	○	○	○	Promptbreeder [28]	✓	○	○	✓
DSPy [75]	—	○	○	✓	ProTeGi [21]	—	○	○	✓
Dynamic Cheatsheet [49]	—	○	○	○	ReasoningBank [38]	✓	○	○	○
Ecdysis [93]	✓	○	○	✓	ReCreate [97]	✓	○	○	✓
EffiSkill [58]	✓	○	✓	✓	Reflexion [8]	✓	○	○	—
EGO-Prompt [26]	—	○	○	✓	RHI [81]	—	○	○	—
Evo-Harness [91]	✓	○	○	○	RoboPhD [98]	✓	○	○	✓
EvolveNet [109]	✓	○	○	✓	RRSI [101]	✓	○	✓	✓
EvoRepair [43]	—	○	○	—	Self-Harness [86]	○	✓	○	○
ExpeL [6]	✓	○	✓	✓	SePO [27]	✓	○	○	✓
FlowEvo [84]	✓	✓	✓	○	SEW [79]	○	○	○	✓
Gödel Agent [111]	○	○	○	✓	SICA [10]	—	○	○	—
GEPA [23]	✓	○	○	✓	Skill Consumption [62]	✓	○	○	✓
GPTSwarm [73]	—	○	○	—	SkillClaw [36]	—	○	○	—
Group-Evolving Agents [105]	—	○	○	○	SkillForge [46]	✓	○	○	✓
GSE [48]	✓	○	○	✓	SkillMOO [57]	—	○	✓	—
GSkill [47]	✓	○	○	✓	SkillOpt [51]	—	○	○	✓
GUI-HARVEST [94]	○	○	○	○	SkillOpt-Lite [52]	✓	○	○	✓
Harness Ceiling [95]	✓	○	○	✓	SWE-Exp [39]	—	○	○	○
HarnessBank [96]	✓	○	○	✓	Symbolic Learning [78]	—	○	—	—
HarnessCompass [88]	—	○	○	—	TACO [56]	—	○	○	—
HarnessFix [87]	✓	○	○	✓	TextGrad [22]	✓	○	○	✓
HarnessX [117]	—	✓	○	—	Trace [24]	✓	○	○	✓
HGM [104]	—	○	○	—	Trace2Skill [35]	✓	○	✓	✓
Janus [61]	—	○	○	✓	TTHE [112]	—	○	○	—
JudgeFlow [80]	○	○	○	✓	VALVE [55]	✓	○	○	✓
Life-Harness [113]	✓	○	○	✓	VASO [59]	—	○	○	—
Live-SWE-agent [85]	✓	○	○	—	VERSE [110]	○	○	○	✓
Living-Harness [54]	—	○	○	—	Voyager [7]	✓	○	○	○
MaAS [76]	○	✓	○	✓					

Sel. selection Act. activation Att. attribution Imp. improvement (task distance).

separating these effects needs a frozen control and periodic retention probes.

These protocols differ in distance, baseline, and the timing of evaluation, but do not by themselves track the later contribution of a particular accepted update. Two questions remain. One is whether capability acquired in earlier rounds survives later ones; Section 6.3.2 takes it up. The other is whether a gain established on held-out tasks is still present several rounds later, at the granularity of the update that produced it. VALVE and PolySkill re-measure an evolving state after the loop has moved on (Section 6.3.2), but neither attributes a later change to a particular accepted update, and none of the tabulated methods does.

6.3 Longitudinal and Dynamic Evaluation

A loop that keeps updating does not add a fifth claim, and it does not make the four compulsory for every study. It changes what each one is asked of. A final evaluation can support the claim that the state reached is better under the tested conditions, and that claim needs no round-by-round record. Claiming that each accepted update was reliable needs evidence at each round, and claiming a lasting gain or retention needs the corresponding later observations, under two kinds of drift: the harness changes with each update, and the task stream or benchmark may change with it. Section 6.3.1 lists the controls the round-by-round claims require and what each one protects. Section 6.3.2 asks what current benchmarks show about state carried across rounds. Section 6.3.3 treats benchmark refresh and the crossed comparison it entails.

6.3.1 Evaluating Successive Updates

When updates follow one another, each transition $H_k \to H_{k+1}$ depends on earlier rollouts, feedback, and admission decisions. Outcomes along this history are therefore not fresh, independent evidence for each later state, and crediting a change to a particular update needs references that do not move with the agent. The first such reference is the model, which Section 2 assumes fixed. Holding M fixed is a property of the study, not of the method. A hosted model may be revised, re-quantized, or re-routed between rounds with no change on the harness side, and a comparison that spans the change credits H_{k+1} with a difference M produced. Pinning a provider version and recording it with the decoding parameters makes the assumption checkable. Where no version can be pinned, the model is an uncontrolled input and the report should say so. Table 6 lists seven controls with the claim each protects and the confound it addresses.

Each control buys a stronger claim at a cost, and each has a limit. Permuting task orders measures sensitivity to order, not transfer. Probes on earlier distributions measure retention only until their results begin to guide later updates. A later re-measurement shows whether a credited gain is still there, but not which update in between removed it. None of the controls restores independence to a task stream that the loop reuses adaptively. Section 5.2 treats the narrower statistical questions under explicit sampling assumptions.

6.3.2 Retained State Across Rounds

None of the four claims is tied to the round that accepted an update, and asking each one at a later round changes it: whether the probes have stayed outside the update loop, whether the admitted artifact is still loaded and still followed, whether the difference can still be separated from an envelope grown since, and whether the credited difference is still there. Current longitudinal benchmarks bear mainly on the activation and improvement questions, and this subsection reads them against those two; the selection and envelope questions fall to the evidence-separation and inference-envelope controls of Table 6.

A sequential benchmark is more informative about the update mechanism when it records the state carried forward alongside task scores. A controlled evolving-versus-frozen comparison can still estimate the effect of the overall learning policy without tracing each retained artifact. Several

Table 6 Controls for evaluating successive updates. Rows are grouped by the claim each control protects. The second column is the minimum a report has to log for the control to be checkable, and the third is the confound that the record makes checkable. Citations point to an instance of the control or to evidence that the confound occurs.

Control	What to record	Confound addressed
Selection: was the number measured or picked?		
Evidence separation [130, 141]	Which tasks and signals were used for proposal, ranking, admission, and reporting, and how the reported round and configuration were chosen.	Reporting evidence reused by the search.
Oracle integrity [96, 138, 139]	That scoring, verification, and bookkeeping code lie outside what a candidate can edit or repeatedly probe.	A candidate changing the measurement that judges it.
Activation: did the update operate in the scored runs?		
Activation record [70, 89, 117]	For each task and attempt in the scored runs, the three rates of Equation (11): the artifact was available, its path ran, and its effect carried.	An update credited although it never operated.
Attribution: did the difference come from the update?		
Execution variation [5, 103, 145]	Repeated runs of the frozen state on the same tasks, or a paired test over per-task outcomes, with the margin set against that spread. Spread over judges or evaluator seeds does not count.	Noise read as progress.
Inference envelope [103, 117, 145, 146]	For the scored runs: generated tokens, model and verifier calls, turns, retries, attempts or candidates sampled, and the reasoning-effort setting. Context added by the artifact and search cost are stated separately.	Added inference compute read as a harness gain.
Model and evaluator record [145, 148, 152]	Model identifier and version, decoding parameters, scorer and judge versions, and any change to them during the study.	A changed model or grader read as a harness gain.
Improvement: how far does the difference extend?		
Controlled histories [153]	Varied task orders, a frozen reference state kept alongside the loop, probes on earlier distributions the loop does not control, and a re-measurement of each credited gain several rounds later.	Order effects read as transfer or retention; a gain measured once read as lasting.

suites evaluate state carried across task sequences, or linked sessions in which earlier writes affect later work [154–158]; these designs expose the dependence between runs, though a later success score alone still does not show which retained artifact was loaded or used. ServeLearnBench, which streams environments whose undisclosed rules change over time, adds a caution from the other side: across learning harnesses and models, continual adaptation had costs and could degrade behavior that was already correct [159].

Activation changes as updates accumulate. Work on retirement, voting-based curation, library drift, and contaminated or self-poisoning updates shows that later revisions change which artifacts remain available and which get selected [160–164], so an activation record taken at the round an artifact was admitted does not carry forward: it does not show that the artifact is still reachable, still preferred over later additions, or still followed once the library around it has grown. Rollback raises the same question, and neither AHE's file-level reverts nor MOSS's image rollback on failed health probes reports whether the reverted state is the one later runs load [89, 108].

Other suites score the artifacts a loop produces, or lengthen the horizon over which it is observed [165–169]. The count, validity, or diversity of artifacts is evidence that state was produced, not that it was activated or that it improved anything, and a long horizon by itself

measures neither forgetting nor accumulated cost.

Ablations of the persistence mechanism give stronger evidence: PAST-Bench and FinEvo-Bench compare runs with persistence enabled and disabled, which shows that persistence changes outcomes under the tested protocol without identifying which stored item caused the difference, since that needs activation records or a more local intervention [170, 171].

Several recent studies measure retained performance directly. VALVE freezes its current bank at step 0 and every ten mixed tasks, scores each frozen checkpoint on the same disjoint 100-task panel with three rollouts per task, and reports peak-to-final drawdown. The peak is selected on that panel, so it is not an untouched final-test estimate [55]. PolySkill revisits its starting WebArena Shopping tasks after adapting to Amazon and Target and ends +4.9% over ASI, in a case study separate from its primary online comparison [83]. Both track the later performance of an evolving state.

Generalization measurements and admission gates answer related but different questions. Agent Plasticity measures how much experience an agent converts into held-out gains [172]; a study of 21 skills that improved their training tasks finds that five retain all of the improvement on held-out tasks, thirteen retain part, and three retain none [173]. Self-Harness and HarnessX enforce non-regression on their current evaluation sets, leaving capabilities untested by those sets outside the guarantee [86, 117]. HarnessBank's retention ratio divides test gain by training gain; it describes transfer and is not a statistical test or a measure of performance after later updates [96].

What remains missing is a later estimate of the causal contribution of a particular accepted update. AHE notes that one repaired task remains solved through later iterations, but this does not isolate the retained contribution of the edit that first repaired it [89]. Such an estimate requires later probes on an earlier distribution, together with a comparison that isolates the credited update within the subsequent harness state. A longitudinal report should distinguish state carryover, later-run activation, retained performance, and update-specific contribution, while also recording rollback, adaptive reuse of evaluation evidence, and cumulative cost.

6.3.3 Benchmark Refresh and Crossed Evaluation

Refreshing tasks limits saturation and direct contamination. It also introduces a second source of change: candidate updates may be exposed to the refreshed instances, and difficulty may vary between releases. Instance exposure and benchmark version therefore have to be recorded separately from harness updates.

Current benchmarks refresh in three ways, and each leaves a different problem. Synthetic generation derives new instances from existing ones or from diagnosed failures, as in Benchmark Self-Evolving and StressEval, where the generator and the candidate update may read the same traces and the report should record that overlap [174, 175]. Time-stamped collection admits tasks released after a cutoff, as in LiveBench, LiveCodeBench, LiveMedBench, SWE-rebench, and SWE-bench Live; a later release date reduces direct overlap but neither equalizes difficulty across releases nor shows that related task patterns were absent from earlier development [143, 144, 176–178]. Adversarial self-play lets the challenge distribution move with the participants, as in MathDuels and the Red Queen Gödel Machine, so scores depend on the participant pool and the verifier, and changes in the harness, the task generator, and the evaluator must be reported separately [179, 180].

None of the three observes the harness update itself, and each adds a second axis of variation that has to be separated from the first: comparing H_k on snapshot $\mathcal{D}_A$ with H_{k+1} on snapshot $\mathcal{D}_B$ mixes the two. A crossed comparison evaluates both harness states on both snapshots under the same scoring procedure, so differences within a snapshot belong to the harness state and differences within a state belong to the snapshot. It is still conditional on the tested tasks, execution protocol, scorer, and inference envelope, and it does not restore independence if either snapshot has already informed candidate generation or admission.

Evaluating every historical harness on every snapshot is usually too costly, so a study can keep a versioned set of anchor tasks across adjacent snapshots and cross-evaluate selected checkpoints, reporting the coverage and replication it chose, since reducing either narrows the differences it can detect.

Section synthesis. Taken together, these protocols keep selection, activation, attribution, and improvement apart as four empirical claims and match each claim to the evidence it needs: an independent final evaluation for the state reached, and per-round activation records, effect separation, and retention probes for what is said about the updates themselves and their lasting effect. Section 5 gave conditions for admission, stability, and retention under explicit assumptions; it does not give a general guarantee for multi-round harness evolution.

7 Applications of Harness Self-Evolution

Harness self-evolution matters in deployment because the experience an agent gathers while working can change how it works later, without retraining the model. This section examines five existing and prospective settings for that capability: personal assistance and developer workspaces, enterprise automation, scientific discovery, robotics and embodied interaction, and public services and clinical systems. For each, we ask what the harness accumulates, how that changes later execution, and what limits it. The settings also differ in how far an update travels and how easily it can be reversed. Workspace artifacts may remain local, enterprise state can spread across teams, scientific results are slow to confirm, physical damage survives software rollback, and public services add legal accountability. These differences shape how each setting admits changes, and they recur in the societal effects of Section 8.

Figure 7 summarizes these five regimes through a shared execution–feedback–validation–reuse loop. The surrounding panels pair each regime's evolving state with its validation requirements, from local tests and memory curation to independent experiments, physical safety checks, and expert approval.

7.1 Personal Assistance and Developer Workspaces

In programming, writing, and local research, an agent meets the same project, the same conventions, and the same user again and again, so experience pays back quickly. The harness accumulates corrections, preferences, project facts, and reusable procedures tied to one workspace [12]. Commercial coding agents now provide this layer. GitHub Copilot keeps repository-scoped memory of coding conventions, architectural decisions, and build commands, uses only entries it has validated against the current code, and deletes entries left unused for 28 days [66]. Claude Code combines user-written instruction files with an auto memory of user corrections, preferences, and project context [64], and Anthropic Agent Skills defines a file-based skill format [67]. Windsurf Cascade and Cursor automations keep memories from earlier sessions and runs [181, 182]; Augment and the experimental Auto Memory of the Gemini CLI propose memory or skill updates for the user to approve [65, 183]; Devin recommends prompt and knowledge edits after a session [184]; Replit, Letta Code, OpenClaw, and Hermes load installed or agent-created skills [185–188]; and Prime Agent applies evidence-backed refinements to its supplemental prompts, memories, and skill descriptions [189]. The documentation shows widespread use of a persistent layer in this category, but it rarely describes how revisions are formed or reports their effect on later work, and user-written instruction files are configuration rather than self-evolution (Section 2).

Research systems show how this layer can be revised from the agent's own experience. Dynamic Cheatsheet and ReasoningBank turn self-assessed task outcomes into a cheatsheet and a bank of reasoning strategies [38, 49]; ACE evolves its context as a playbook from execution feedback, and Trace2Skill consolidates trajectories into skill directories [35, 50]; SkillOpt and SkillOpt-Lite

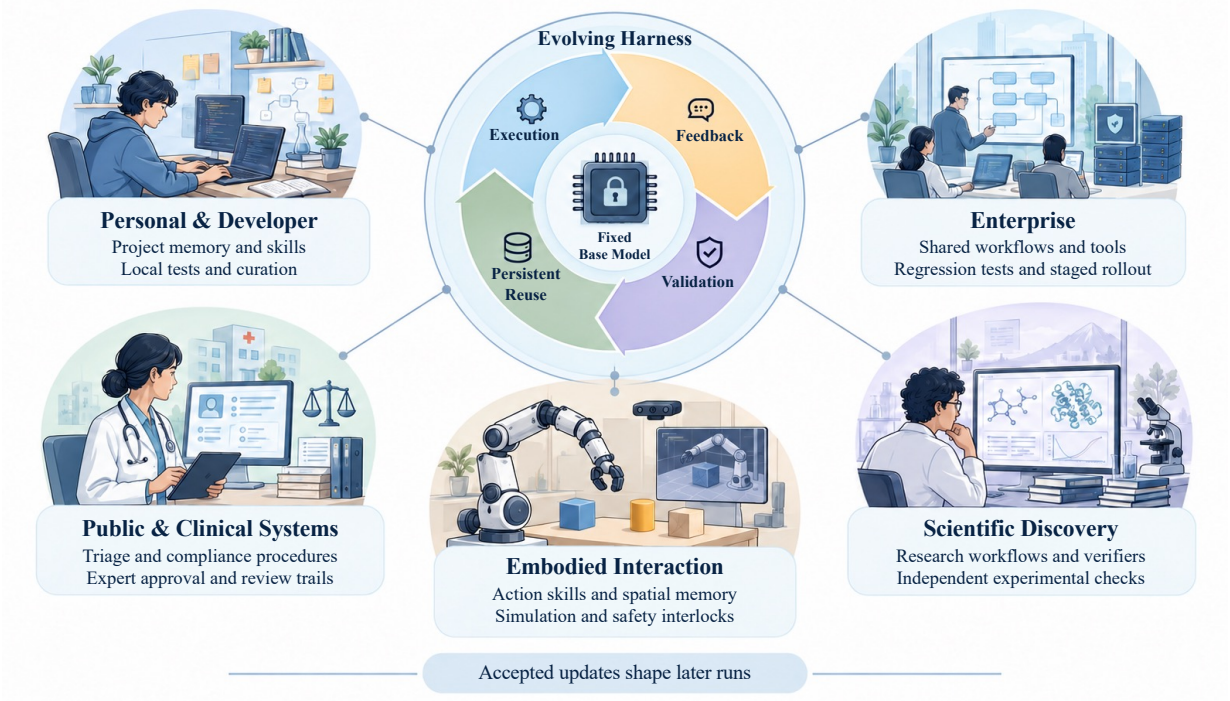


Figure 7 Applications of harness self-evolution across five deployment regimes. With base-model weights held fixed, execution feedback informs harness revisions that are validated, retained, and reused in later runs. The surrounding panels identify representative update targets and validation requirements for personal and developer workspaces, enterprise automation, scientific discovery, robotics and embodied interaction, and public services and clinical systems. These regimes share an update lifecycle but differ in how state is shared, how revisions can be verified, and whether external actions can be undone. The figure summarizes deployment requirements rather than establishing improvement for every application.

refine an individual skill and keep an edit only if it passes validation [51, 52]. A useful saved skill can spare later runs from rediscovering an action or tool-call sequence; some systems report fewer steps per task on their agent benchmarks [190, 191].

That benefit depends on the saved state being used well, and saving a skill does not show that it operated in a later run (Section 6.2.2). An admitted skill may not be loaded, or may be followed less as a trajectory grows, especially with weaker models [70]. Self-generated skills do not substitute for curated ones, and focused skill sets outperform exhaustive bundles [69], so a library that only grows can fill with files that do not help. Instructions planted through untrusted inputs or malicious interaction trajectories also persist into later sessions [71, 72, 192]. Reported safeguards include a regression check that compares old and new memory on a small task set [61], validation and expiry of stored entries [66], and user approval of proposed memories and skills [65, 183]. Local versioning can reverse an artifact edit, but it cannot necessarily reverse a file deletion, disclosure, or API action performed while that artifact was active. A continuing practical question is curation over time, that is, which accumulated items to merge, retire, or keep as the library grows.

7.2 Enterprise Automation

In IT operations, customer support, and collaborative software engineering, knowledge of how to handle a failure is spread across individuals and teams, and the systems it concerns keep changing. Harness self-evolution turns that scattered experience into shared skill libraries, triage rules, and repository middleware [11, 12] that can be updated as internal APIs, dependencies, and business rules change. In customer and IT support, SkillForge diagnoses execution failures in batches, locates the deficient skill, and rewrites it [46], and SkillClaw pools trajectories from many users

into a shared skill library [36]. In software engineering, DarwinX, AHE, MOSS, and Ouroboros update executable tools, middleware, and control logic [89, 103, 107, 108].

Shared state can turn one team's lesson, or mistake, into another team's procedure. A patch, retrieval filter, or shortcut prompt admitted into shared state reaches every user and service that loads it, and a shortcut that raises a measured proxy while degrading the process it stands for is a form of reward gaming [193, 194]. Enterprise systems handle this with practices borrowed from software engineering. DarwinX keeps a child harness only if it improves on some task without losing more than a small tolerance of what its parent solved [103]; harness patches can be validated against regressions before they are accepted [87]; and MOSS and Ouroboros test candidates in isolated containers, apply health checks, and land changes as reviewed commits that can be rolled back [107, 108]. A shared update also has to work for teams whose tasks did not produce it, so testing across teams is part of what an enterprise loop must provide.

7.3 Scientific Discovery

Scientific and technical work runs in campaigns that span literature search, code, simulation, and data analysis over many sessions [14, 195]. The harness accumulates research know-how: experimental setups, tool settings, search heuristics, verification checks, and the record of what failed [118, 196], and this state must stay reproducible across runs [197]. CORAL keeps shared notes, tool settings, and search heuristics during open-ended optimization [196], and Argus stores memories, workflows, and routing rules in durable campaign state through role-based review [118]. AlphaEvolve evolves external programs and mathematical heuristics against automated evaluators [198]; it revises the artifact under study rather than its own harness, which makes it an adjacent case in the sense of Section 2. PSI's vision of an AI-native physics laboratory extends the campaign structure to virtual scientists that run hypothesis generation, simulation, and live facility monitoring bounded by conservation laws and digital twins [199].

The difficulty in this setting is that results are slow and costly to confirm. Autonomous research agents declare success on implementations and evaluations that in fact failed [200], and case studies of a frontier model in mathematics and science record confident errors that only expert checking caught [201]. A self-evolving research harness can also change its own checks: if it admits flawed verification code or a loosened test, later results look better without being better. This is a failure of oracle integrity, the case of Section 6.2.1 in which a candidate changes the record used to judge it, and it also leaves the gain unattributable to the harness (Section 6.2.3); a campaign can then cycle through internally consistent but invalid findings [193]. Accumulated know-how keeps its value when the final checks stay outside what the loop may edit, as execution-based correctness tests, held-out benchmarks, and physical experiments do [202, 203]. Section 8.3 returns to what happens when many campaigns share the same capacity to confirm results.

7.4 Robotics and Embodied Interaction

Robots and other embodied agents face objects, hardware, and contact conditions that change between deployments in ways a fixed skill library does not anticipate. RoboCat addresses this variation by retraining a generalist model, which places it outside the scope of this survey, whereas code-based skill transfer adapts the skills themselves [204, 205]. Harness self-evolution leaves the perception–action backbone fixed and changes what surrounds it: executable action skills, API wrappers, spatial memory, and recovery procedures [12]. EMHO revises how the agent monitors progress, uses vision tools, grounds observations, and recovers from failure, from execution traces alone [206]. RoboRSI coordinates subagents around an API-accessed vision-language model to maintain a task–skill tree, turning successful manipulation runs into executable code that must pass a no-regression check in simulation before it is committed [207]. LYRA stores human-approved manipulation skills [208], Memory Transfer Planning adapts code-based skills from simulation to new robot hardware [205], and MineEvolve, in an open-world game environment, derives reusable

skills from successful executions and remedies from failed or stalled ones [209]. EmbodiedSWE is an adjacent case: coding agents solve long-horizon dexterous manipulation tasks in simulation, and the verified solutions become training trajectories for a robot policy, so what persists is policy weights rather than revised harness state [210].

Software rollback cannot undo physical damage. Reverting the harness restores the skill library but not a collided part, worn hardware, or a damaged workspace, and safety evaluations of tool-using agents count property loss and physical harm among their failure categories [211]. Skills are therefore checked before they act, as RoboRSI's simulator check does [207], and the constraints a skill must respect can be written as specifications enforced on tool actions before they execute [212]. Most skills are also formed in simulation. Memory built in simulation has carried over to hardware experiments [205], but the gap between simulated and physical conditions means that a skill passing in simulation still needs staged trials on hardware, with emergency stops and human overrides kept outside what the loop can change.

7.5 Public Services and Clinical Systems

Clinical consultation, public health surveillance, legal review, and administrative services operate under legal accountability and professional standards, and the guidance they follow changes as rules, guidelines, and case law change. Frequent fine-tuning can incur retraining and revalidation costs, introduce forgetting [213], and make individual behavioral changes difficult to inspect. Harness updates offer a possible alternative: triage criteria, drug reaction monitors, and compliance checklists are human-readable and can be revised and reviewed directly while the weights stay fixed. Fixing the weights keeps one component unchanged; it does not by itself preserve the validation or regulatory status of the system, because the rules and workflow around the model have changed. The evidence is still early. Agent capability in these domains is measured on legal-reasoning and professional-task benchmarks [214, 215]; MedAgentBench provides an interactive FHIR-based environment for clinical record tasks [216], AgentClinic simulates clinical dialogues built from EHR-derived cases [217], and LiveMedBench supplies time-stamped medical evaluation streams intended to limit contamination [178]; MCE adapts external context and skills on symptom-diagnosis and legal benchmarks [53]. These are benchmarks and research prototypes, and they do not yet show a complete update loop in a deployed public service.

Errors in these settings take the form of incorrect treatments, statutory violations, and unlawful administrative decisions, and they fall on people who did not choose the system. Writable memory admits poisoned entries [72], and safety behavior can decay as an evolving agent accumulates memory [218], so a harness can drift from the rules it was validated against without any single update looking wrong. A workable division of labor lets the system draft candidate triage patterns or compliance rules from recent operational failures, while qualified professionals decide, after testing on cases the loop did not see, which of them enter use. The record of those decisions is also what later oversight relies on (Section 8.5).

8 Societal and Economic Implications

When agents keep what they learn, experience stops being a by-product of work and becomes an asset that can be stored, shared, and copied. This section considers what that changes beyond a single deployment: how routine work and skill formation shift when procedures are reused (Section 8.1); who captures the value of pooled experience and what shapes competition (Section 8.2); how institutions retain knowledge and what bounds the capacity of research and public services (Section 8.3); what happens when errors and sensitive information enter persistent, portable artifacts (Section 8.4); and how explicit, revisable rules change discretion and accountability (Section 8.5). Longitudinal evidence on these effects is scarce, so the section describes mechanisms and the conditions under which they would hold rather than estimates.

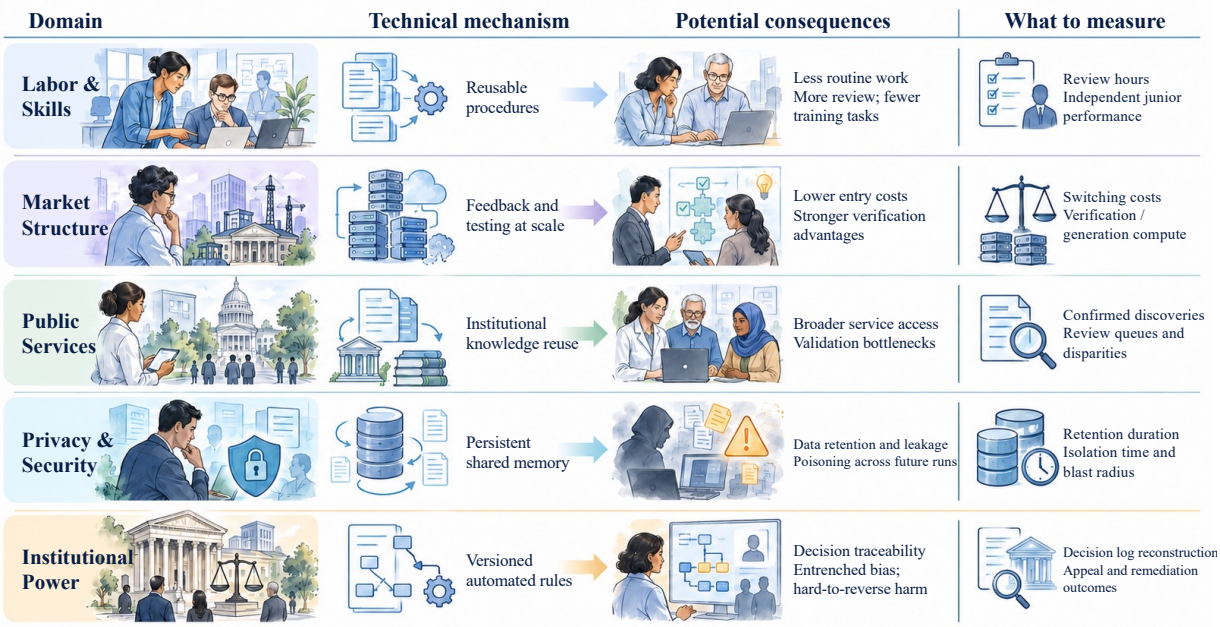


Figure 8 Societal and economic implications of harness self-evolution. The five rows connect reusable procedures, feedback and testing at scale, institutional knowledge reuse, persistent shared memory, and versioned automated rules to potential effects on labor and skills, market structure, public services, privacy and security, and institutional power. The rightmost column lists indicators for empirical assessment, including review effort, switching costs, verification capacity, data retention, and review and remediation outcomes. Arrows denote hypothesized pathways rather than established causal estimates; the balance of benefits and harms depends on deployment conditions and governance.

Figure 8 organizes this analysis into five domains, linking the technical mechanisms of persistent harness evolution to potential societal consequences and observable indicators.

8.1 Productivity, Work, and Education

A reusable procedure records the result of a diagnosis. Diagnosing unfamiliar dependencies, tuning API parameters, and finding a valid control flow are costs incurred on many tasks, and a playbook or routing rule pays them once and replays the result [89, 118]. What the replay saves is the discovery, not the execution.

Whether such savings add up to productivity at the level of an organization has been measured for adjacent tools but not for harness self-evolution. Liu and McKinnon study GitHub Copilot adoption against participants' historical engineering metrics and report an efficiency–throughput gap: per-developer efficiency gains do not appear as measurable organizational throughput [219]. Earlier task-automation accounts and per-task Copilot studies describe adjacent settings rather than this one [215, 220–222]. The mechanism does suggest a shift in what human work consists of: reusable procedures can automate parts of routine synthesis, while problem formulation and failure verification still require task-specific judgement and external evidence, even when tools assist them.

The same absorption bears on how professionals acquire skill. Junior staff develop diagnostic judgement through routine debugging, draft writing, and minor failure triage, and these are the exploratory stages a harness loop turns into reusable artifacts. The staff who would later review harness proposals and runtime failures acquire that judgement by doing the work the harness now does. A randomized experiment with 52 developers learning an unfamiliar library found significantly lower subsequent quiz scores with AI assistance and no significant average speed-up. Within the assisted group, cognitively engaged interaction patterns were associated

with better learning; this subgroup pattern was observational [223]. The direction is not fixed, since verification work is itself a training ground and a loop that surfaces its own failures may teach diagnosis faster than unaided debugging did.

This effect has been measured most directly in education. In a field experiment with nearly a thousand high-school mathematics students, relative to the no-AI control, a standard GPT-4 interface raised grades during assisted practice by 48% and lowered them by 17% once access was removed, whereas a tutor built on the same model with prompts designed to safeguard learning raised assisted grades by 127% and largely removed the later deficit [224]. The two arms differ in configuration rather than in model, which places the effect at the layer a harness update edits. Laboratory and field studies likewise find that using an LLM to substitute for learning activities reduces understanding while complementary use increases it [225], and a tutor with structured prompts and scaffolding outperformed in-class active learning in an undergraduate physics course [226].

These fixed-configuration studies motivate a concern for self-evolving tutors: revisions that raise assisted performance may also change what learners can do on their own. Because the unguarded configuration improved assisted practice and still left students worse off once access was removed, an update target based only on assisted performance would not register the loss, and the effect on independent ability has to be part of what such a tutor is updated toward. Research tutors already accumulate cross-session learner memory or optimize their tutoring prompts without changing model weights, but the systems we located score their updates against simulated students [227–229], so whether those updates help or harm the unassisted learning of real students has not been tested. Whether effects of this kind hold at population scale is not established by existing measurement [230]. Useful indicators include human review hours per completed workflow, task lead times [231], and the unassisted performance of junior staff and learners once harness tools are removed.

8.2 Economic Distribution and Market Structure

Who gains from these savings depends first on who holds the artifacts. The traces a harness learns from are produced by employees and users, while an operator may control access to and distribution of the playbooks and skills distilled from them; legal ownership depends on the applicable contracts and rules. Pooling many users' trajectories, as in SkillClaw [36], can concentrate the value of that reusable experience at the platform. Accumulated state also binds users to a platform. An organization that has built up procedures, memories, and tuned workflows on one system loses part of that investment if it moves, unless the artifacts can be carried along, which is why the format of harness state, such as file-based skills [67], has economic weight as well as technical weight.

Competition also depends on what it costs to improve a harness. Because base models stay fixed, specialized firms can reach strong task performance by adapting harnesses on domain workloads without the capital cost of pre-training. Proposing a workflow or code mutation can cost less than checking it for silent regressions through a test suite, a container farm, or a physical trial [199, 202, 203]. Incumbents with continuous user traffic collect more failure traces and can afford more of this checking, so they may improve their harnesses faster than smaller actors can without accepting more regressions or reward gaming [193, 194]. Against this, verifiers are software and can be shared. Execution-based benchmarks release the apparatus rather than only a score, across repository tests, GPU-kernel correctness and speed, and machine-learning graders [5, 203, 232]; automated collection pipelines keep such suites renewable instead of fixed [143, 144]; and the benchmarks built for harness optimization supply held-out partitions and resource accounting [130, 131]. Shared verifiers lower part of the cost of entry, though not the cost of running them at scale, of private test data, or of the staff who maintain them. Whether accumulated experience and checking capacity concentrate the market, or portable formats and

shared verifiers keep it contestable, is an open empirical question; switching costs between harness platforms and the performance gap between instrumented enterprise agents and open baselines are the most direct indicators.

8.3 Innovation and Public-Service Capacity

Persistent harness state changes how institutions keep knowledge. In long scientific and engineering projects, progress often stalls when key personnel leave or unwritten lab conventions fade; experimental parameters, tool tricks, and error-recovery paths kept in durable, versioned repositories can preserve continuity across long-running campaigns [118, 196, 197]. Knowledge kept in artifacts is not necessarily kept by staff, however, which ties institutional continuity to the skill-formation concern of Section 8.1. In public administration, healthcare intake, legal aid, and public education, the same persistence could let resource-constrained agencies offer procedural assistance at lower marginal cost. Benchmarks for clinical and legal agent tasks exist [214, 216, 217], but effects on waiting queues and service reach have not been measured, and the evidence on learning is discussed in Section 8.1.

Producing more candidates does not proportionally expand the capacity to confirm them. Hypotheses, legal briefs, and clinical plans can be generated at the rate of model calls, while the confirmation they need is a wet-lab test, a clinical trial, or a hearing, whose capacity does not grow with the number of candidates [199]. EmbodiedSWE shows the same asymmetry in robotics: coding agents generate many verified simulation trajectories, while transfer to real-robot execution passes through a limited physical channel [210]. Self-evolving systems may improve triage, scheduling, or experimental design, but additional proposals alone do not expand physical or legal verification capacity. Confirmation may therefore take a growing share of research and service costs. Where the signal a system learns from is a loose proxy, it can adopt shortcut rules, and reward hacking under strong optimization can become hard to detect [218, 233]; in a public service, such rules could propagate administrative errors before review catches the drift. Whether these systems add capacity or mainly add candidates will take long-term observation to establish, and appeal rates and error rates across the populations a service reaches are among the signals worth tracking.

8.4 Privacy, Security, and Systemic Stability

Harness updates are distilled from execution traces, tool telemetry, and user inputs, and whatever they retain can be read, subpoenaed, sold, or leaked. API credentials, business logic, and private user details can therefore persist in playbooks or memory indexes long after the session that supplied them ended. Studies of injection through memory and instruction files show that content written there stays active across sessions [71, 72], and nothing in that mechanism distinguishes an attacker's payload from a user's credentials. Where a skill library is pooled across users and built to propagate updates through the system [36], patterns learned from one user's workflow can enter the context of colleagues who were not authorized to see the underlying data. One consequence is definite: once user data has been distilled into a behavior heuristic or a tool script, deleting the raw input does not remove its influence on later behavior, so honoring a deletion request does not guarantee deletion. Closing that gap requires recording which artifacts were derived from which inputs, because the influence resides in an artifact that may no longer contain an identifiable copy of the input.

Persistence also extends the life of an attack, and checking that a learned skill succeeds on tasks does not reliably stop it. Malicious interaction trajectories can induce an agent to generate skills that are then persistently stored and repeatedly activated [192], and payloads planted in memory files attack current and future sessions [71]. Poisoning does not even require a malicious trajectory: successful experiences that are each task-correct and pass verification can be assembled so that the distilled skill applies a target behavior outside the conditions that constrained it, with an

attack success rate of 95.71% in the reported setting [234]. In self-evolving coding agents that author new tools by imitating skills retrieved from a shared library, a malicious skill can become the template for new skills that keep its payload, and the copies re-enter the library and persist after the planted skills are removed [164]. The risk is not limited to adversarial input. SEABench pairs evolving agents with non-evolving ones on the same 48 task sequences: self-evolution raised task completion and, in doing so, produced safety failures the paired baseline did not have, with the divergence visible in the agents' own reasoning [235].

A shared base model already gives its deployments a common source of error, the setting that analyses of algorithmic monoculture study: decision-makers that share a component also share its errors, and the same individuals receive the same adverse outcome everywhere [236, 237]. Portable harness artifacts add a second path alongside the shared model, because skills, tool servers, and memory files can be copied, installed, combined, and regenerated independently of the model and its provider. Skills are installed from public registries such as ClawHub and the Hermes Skills Hub [187, 188], and measurements of these registries find flawed artifacts at scale: 26.1% of 31,132 marketplace skills contain at least one vulnerability pattern [238], over half of 157 confirmed malicious skills came from a single actor [239], and malicious tool servers passed the audits of three aggregation platforms [240]. Use is concentrated as well. On ClawHub the top tenth of skills receives 46.93% of downloads while three security scanners disagree on 23,702 of 61,990 skills [241], and servers in the Model Context Protocol ecosystem share dependency monocultures [242]. Content written into shared memory is retrieved for other users' queries [243], and self-replicating prompts spread across connected applications [244]. Together these findings describe a route by which one faulty or malicious artifact is loaded by many deployments at once.

The reviewed studies do not directly measure the resulting cross-organization failures. We state as a hypothesis that a self-evolving harness that admits or publishes such artifacts can turn a local fault into correlated failures across organizations, and that this risk grows with the concentration of verification discussed in Section 8.2. Exposure to such a failure can be tracked by the number of deployments that load a given artifact version, and its duration by the time needed to detect and quarantine a poisoned skill.

8.5 Governance and Institutional Power

Harness state is explicit: the operational state consists of code, prompts, and configuration files that can be versioned, diffed, and read. This gives governance a finer interface than a fine-tuned weight update offers, but it is a capability, not a guarantee. An erroneous decision can be traced to the rule that produced it, the execution trace, the test benchmark, and the gate approval that admitted the rule only when the deployment records versions, run traces, and inputs and outputs [152], and only when the state a run loaded is the one on record. Even then the record shows which rule ran, not that the rule caused the decision. Traceability therefore depends on the operator's instrumentation as well as on the representation.

Such a record also cannot undo an action. Reverting the harness to an earlier commit leaves in place an improper financial transfer, a corrupted medical record, or a damaging actuator movement that the erroneous state produced, just as runtime rollback restores in-process state but not effects already emitted, such as a sent message [116]. Accountable deployment needs both versioned records, which explain an outcome after the fact, and runtime write barriers, staging environments, and compensation or remediation mechanisms, which act before the harm or compensate for it.

The remaining effect concerns discretion. Human case workers interpret ambiguous regulations with situational context, while an evolving harness encodes past operational choices as retrieval rankings, eligibility filters, and routing policies [245]. Where the execution data reflects organizational bias, self-evolution can reproduce it in inspectable form, which moves the locus of administrative judgement toward whoever configures the admission gate. Inspectability can therefore expose such bias or entrench it, depending on who can read the record and contest what

it shows: affected individuals, their representatives, and auditors need access to the rules that decided their case, and a way to have a contested rule revised.

9 Future Directions

Current systems can already revise prompts, memory, skills, workflows, and code from their own experience. Making such revisions accumulate over long operation remains unsolved: they have to keep working together as the state grows, adapt as tasks and environments change, and stay deployable and governable once they are shared. We group the open problems into six directions: co-evolution of models and harnesses; search over heterogeneous harness state; infrastructure for growing, shared, and concurrently modified state; evaluation of long-running evolution; safety boundaries for systems that modify themselves; and governance of who may change, approve, distribute, and withdraw evolved state.

9.1 Model–Harness Co-Evolution

This article fixes the base model to isolate harness-level updates, but deployed systems may also update weights, and recent work evolves the harness and the weights together [246]. Three questions are open: which capabilities should be internalized into weights and which kept in inspectable harness code; how to schedule updates when model training is slow and expensive while harness adaptation is continuous; and whether an evolved harness becomes specialized to the base model it was evolved with. One study offers task-specific evidence about the first question: on a long-horizon planning benchmark, harness evolution repaired process failures such as blocked calls, loops, and exhausted step budgets, while poor delivered plans were left to weight training, and adapters trained on evolved-harness trajectories carried the harness gain into the weights [95]. The same study also shows where the harness gain lives: on a second benchmark it survived in what the model sees, and training on the same trajectories did not add to it.

For the second question, alternating harness search with policy training, and matching training trajectories to the runtime that produced them, keeps a small corpus from losing transfer when the harness changes [247], and traces produced under a fixed model can serve as training data for later models [81]. On the third, what has been measured is pairing sensitivity rather than co-specialization. Across 66 model–harness configurations, model rankings reverse between harnesses, and a model's own vendor harness is not reliably its best [248], while harnesses found by search on one model have transferred to unseen base models [100]. The reviewed studies do not establish whether a harness evolved on one checkpoint retains its gain on that model's successors; the test would evaluate the frozen evolved harness on later checkpoints against a harness re-evolved on each. None of these studies yet gives a routing rule: for a failure observed in deployment, whether to address it by a harness revision or in the next round of training, and how to tell afterwards which of the two produced a gain.

9.2 Optimization Algorithms for Heterogeneous Harness Search

Current methods combine trace-guided proposals and candidate selection in different ways, including regression gates [117] and populations ranked by benchmark scores [102, 249]. Section 5 relates these derivative-free searches to program optimization [250] and covers the selection mechanisms already in use, including archive-based and quality-diversity search and Thompson sampling over lineages, and adaptive allocation of a fixed compute budget across populations has been studied as well [251]. Four problems remain open. The first is choosing what to update: a failure can be addressed by editing a prompt, adding a memory entry, writing a skill, or changing code, and most methods fix the target in advance. The second is representation, since these objects differ in how an edit changes behavior, yet current operators treat them separately or all as text. The third is composition. Edits admitted at different times interact, a score over a

batch of tasks does not say which edit produced a change or whether two edits conflict, and a library that only accumulates eventually needs merging and retirement. The fourth is cost: search budgets go to generating and scoring candidates, while the cost of confirming a change, which dominates in several settings of Section 7, rarely enters the objective. Progress on these problems would show in comparisons that report, besides final scores, the number of candidates evaluated and their compute cost, variance across seeds, and backward transfer for earlier capability and forward transfer to new tasks [153].

9.3 Infrastructure for Evolving State

As harness state grows and is shared, it needs the kind of management that code repositories and databases receive. Harness RSI adds lifecycle costs across proposal generation, candidate verification, deployment, and rollback [108, 116], and several systems problems recur. A state that many runs read while the loop writes to it has to be versioned so that each run sees a consistent snapshot. Concurrent updates from different users or nodes have to be merged into shared skill libraries [36], including nodes that are intermittently offline. The version a run loads has to be the version that was checked, or records of what ran become unreliable. Verification caches need to identify the harness version together with the model, evaluator, dependencies, task set, and environment, and be invalidated when a relevant condition changes. A new state should be promoted atomically; hot reloading is one way to update runtime components without restarting sessions [116]. Where these operations run is a design choice. One option keeps cached skills and user interaction on client devices and moves candidate synthesis and regression suites to shared servers, which trades latency, bandwidth, device energy, and data residency against the cost of central verification.

9.4 Dynamic Evaluation and Benchmark Refresh

Judging whether long-running evolution helps requires benchmarks that stay informative while the loop adapts to them. Repeated use of a benchmark makes its feedback part of the search [136, 252], and Section 6.3.3 describes how refreshed benchmarks limit contamination while introducing shifts in difficulty, and proposes keeping anchor tasks across snapshots. Three questions remain open. The first is comparability across versions. LiveBench samples tasks, calibrates difficulty, and keeps scoring independent of model judges [176], and LiveCodeBench, Terminal-Bench, and LiveMedBench rotate tasks by release date or collection date [177, 178, 253], but comparability across refreshed snapshots remains unresolved for a continuously adapting harness. An open design problem is how small an anchor set can be, and when anchors should be replaced before repeated use makes them part of the search. The second is the form of a refresh: a refresh that changes difficulty factors confounds capability gains with task shifts unless those factors are held fixed or recorded [175]. The third concerns automated judges, which a self-evolving loop optimizes against repeatedly. LLM judges overestimate agent success and can reverse expert rankings of agents [149], and a judge used inside iterative self-improvement drifted toward maximal scores and favored longer outputs [254]; how such drift interacts with repeated harness optimization remains unresolved in the reviewed protocols. Benchmark tracks for self-evolving systems would also need to record task order, compute budgets, and rejected candidates, and to report specification gaming [150] and safety regressions [218] as outcomes rather than anomalies.

9.5 Safety Boundaries for Self-Modifying Systems

A system that can modify its own harness can also modify the boundaries the harness enforces. Permission settings, tool allowlists, and safety checks are themselves harness state, so which parts of the harness the loop may edit has to be fixed outside the loop, as Section 7.3 argued for the checks that judge research results. Alignment methods applied to the base model, such as Constitutional AI [255], can influence behavior but do not by themselves enforce an immutable

boundary on runtime permissions. The threats of Section 8.4 also reach it from outside: a poisoned skill can be distilled from experiences that each pass verification [234], and a malicious skill can be copied into new ones through imitation [164]. Defenses act at three points. When a skill is learned, a check could test the conditions under which it applies, not only whether its source experiences succeeded; whether such a check detects the context-removal attack of SkillPoison is open. At runtime, least-privilege scoping, time-bounded capability leases for unverified tools, and information-flow constraints between untrusted task memory and privileged middleware limit what a learned artifact can reach [212]. In physical and financial domains, pre-actuation invariants, dry-run shadow execution, and spending limits are needed because rolling back the harness does not recall effects already emitted [116]. Defenses can be compared on attack success rate, time to isolate a corrupted skill, and overhead on benign tasks; a prompt-level counter-defense against EVOMAL lowers its attack success rate to at most 6.7% in the reported settings [164], which gives one reference point.

9.6 Governance, Provenance, and Human Oversight

Evolved harness state raises governance questions that a fixed configuration does not: who may propose an update, who approves it, who may publish it to others, who can withdraw it once it has spread, and how responsibility is divided among them when an update causes harm. In high-stakes institutions, the inspectability discussed in Section 8.5 has to become a procedure. Because an agent's active state reflects a history of automated updates, oversight at scale cannot rely on a person inspecting every step. One open problem is an escalation policy that names observable runtime conditions, such as evaluator disagreement, distribution-shift alerts, or irreversible tool calls, under which autonomous execution stops and authorization passes to a person. Self-correction is not a substitute, since language models struggle to correct their reasoning without external feedback [256], while monitoring an agent's reasoning can expose reward hacking that outcome checks miss [233]. A second problem is the record. Machine-readable evolution trajectories would log each state transition with its triggering feedback, candidate diff, and approval [152, 257], and for shared artifacts the record would have to follow an artifact across organizations, so that the deployments loading a faulty version can be found when it is withdrawn. Whether such procedures work can be judged by how quickly, and how often, the state behind a contested decision can be reconstructed.

10 Conclusion

Harness self-evolution lets agents turn execution experience into persistent revisions of instructions, memory, workflows, tools, and runtime code. We defined this operational feedback loop and organized the literature by what a proposal modifies, distinguishing payloads interpreted by fixed host components from changes to executable implementations and their composition. Holding the base model fixed isolates the harness side of adaptation; the explicit representation of harness state makes revisions inspectable and versionable. Neither property by itself establishes improvement.

The central evaluation problem is that scores both guide the update loop and serve as evidence about its result, yet a task score alone does not show which accepted update ran or what it changed. We separated selection, activation, attribution, and improvement, operationalized them as evidence criteria, and read each reviewed method's primary comparison against them. Gains on tasks held apart from the update are commonly reported; execution records from the scored runs, and margins measured against execution variation, rarely are. An independent final evaluation can establish the performance of the state reached without tracing every component. What an individual update contributed, and whether that contribution lasts, is a different claim that needs execution records, controlled comparisons, and later probes.

The theoretical analysis connects common search structures to evolutionary paradigms and

separates empirical selection from generalization and retention. A finite-candidate argument gives sufficient conditions for an admission comparison to certify improvement: candidates must be fixed before fresh validation tasks are drawn, with error controlled across the comparisons made. Reusing reporting evidence requires an analysis that accounts for adaptation; repeated executions reduce run noise but do not add independent tasks. These conditions clarify what an evaluation supports without guaranteeing the reliability of a deployed loop or the correctness of its evaluator.

Deployment makes the cost of a wrong update concrete. A revision can remain within one workspace or spread through shared libraries, and reverting its files does not undo external actions already taken. Validation and oversight must therefore reflect both the update's reach and the consequences of failure. Persistent harness state can preserve institutional knowledge and reduce repeated discovery, while also carrying sensitive information, unsafe procedures, and errors across users and organizations. The research challenge is to make reusable revisions accumulate without losing earlier capabilities, compromising evaluation, or obscuring responsibility for the behavior they produce.

References

[1] Irving John Good. Speculations concerning the first ultraintelligent machine. In Franz L. Alt and Morris Rubinoff, editors, *Advances in Computers*, volume 6, pages 31–88. Elsevier, 1965.

[2] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements, 2003. arXiv:cs/0309048.

[3] Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. URL https://arxiv.org/abs/2507.21046.

[4] Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems, 2025. URL https://arxiv.org/abs/2508.07407.

[5] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.

[6] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642. AAAI Press, 2024. doi: 10.1609/aaai.v38i17.29936.

[7] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024, 2024. URL https://openreview.net/forum?id=ehfRiF0R3a.

[8] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

[9] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, Singapore, 2025. URL https://openreview.net/forum?id=z5uVAKwmjf.

[10] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent, 2025. URL https://arxiv.org/abs/2504.15228. arXiv preprint.

[11] Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in LLM agents: A unified review of memory, skills, protocols and harness engineering, 2026. URL https://arxiv.org/abs/2604.08224.

[12] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, Lingjie Chen, Yanjun Zhao, Ke Yang, Bingxuan Li, Cheng Qian, Gaotang Li, Xiao Lin, Zhichen Zeng, Ruizhong Qiu, Sirui Chen, Yifan Sun, Xiyuan Yang, Ruida Wang, Rui Pan, Chenyuan Yang, Dylan Zhang, Liri Fang, Zikun Cui, Yang Cao, Pan Chen, Dorothy Sun, Ren Chen, Mahesh Srinivasan, Nipun Mathur, Yinglong Xia, Hong Li, Hong Yan, Pan Lu, Lingming Zhang, Tong Zhang, Hanghang Tong, and Jingrui He. Code as agent harness, 2026. URL https://arxiv.org/abs/2605.18747. arXiv preprint.

[13] Zhe Ren, Yimeng Chen, Dandan Guo, Guowei Rong, Tonghui Li, R. B. Xiong, Qingfeng Lan, Wenyi Wang, Li Nanbo, Yibo Yang, Mingchen Zhuge, and Jürgen Schmidhuber. Self-improvements in modern agentic systems: A survey, 2026. URL https://arxiv.org/abs/2607.13104.

[14] Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops, 2026. survey; arXiv preprint.

[15] Hao Zhou, Haichuan Hu, Tianyu Luo, Ye Shang, Chunrong Fang, Zhenyu Chen, Liang Xiao, and Quanjun Zhang. Self-evolving coding agents, 2026. arXiv:2608.03392.

[16] DeepSeek-AI. DeepSeek harness developer preview. https://www.deepseek.com/harness/en/, 2026. Product/developer-preview page.

[17] Douglas B. Lenat. Eurisko: A program that learns new heuristics and domain concepts. *Artificial Intelligence*, 21(1–2):61–98, 1983.

[18] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.525. URL https://aclanthology.org/2024.emnlp-main.525/.

[19] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=92gvk82DE-.

[20] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=Bb4VGOWELI.

[21] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.494. URL https://aclanthology.org/2023.emnlp-main.494/.

[22] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639(8055):609–616, 2025. doi: 10.1038/s41586-025-08661-4.

[23] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=RQm2KQTM5r. Oral.

[24] Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs, 2024. URL https://arxiv.org/abs/2406.16218. NeurIPS 2024.

[25] Guowei Xu, Mert Yuksekgonul, Carlos Guestrin, and James Zou. metaTextGrad: Automatically optimizing language model optimizers, 2025. URL https://arxiv.org/abs/2505.18524.

[26] Yang Zhao, Pu Wang, and Hao Frank Yang. How to auto-optimize prompts for domain tasks? adaptive prompting and reasoning through evolutionary domain knowledge adaptation, 2025. URL https://arxiv.org/abs/2510.21148.

[27] Wangcheng Tao, Han Wu, and Weng-Fai Wong. SePO: Self-evolving prompt agent for system prompt optimization, 2026. URL https://arxiv.org/abs/2606.04465.

[28] Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 13481–13544. PMLR, PMLR, 2024. URL https://proceedings.mlr.press/v235/fernando24a.html.

[29] Tao Tao, Guanghui Zhu, Lang Guo, Hongyi Chen, Chunfeng Yuan, and Yihua Huang. DelvePO: Direction-guided self-evolving framework for flexible prompt optimization, 2025. URL https://arxiv.org/abs/2510.18257.

[30] Anirudh Nair, Adi Banerjee, Laurent Mombaerts, Matthew Hagen, and Tarik Borogovac. Tournament of prompts: Evolving LLM instructions through structured debates and elo ratings, 2025. URL https://arxiv.org/abs/2506.00178.

[31] Anthropic. Effective context engineering for AI agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025.

[32] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models, 2025. URL https://arxiv.org/abs/2507.13334.

[33] Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 17490–17502, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.866.

[34] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR, PMLR, 2025.

[35] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills, 2026. arXiv:2603.25158.

[36] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. Skillclaw: Let skills evolve collectively with agentic evolver, 2026. URL https://arxiv.org/abs/2604.08377. arXiv preprint.

[37] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, Ge Zhang, Jiaheng Liu, Xingyao Wang, Sirui Hong, Chenglin Wu, Hao Cheng, Chi Wang, and Wangchunshu Zhou. Agent KB: Leveraging cross-domain experience for agentic problem solving, 2025. URL https://arxiv.org/abs/2507.06229.

[38] Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=jL7fwchScm.

[39] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution, 2025. URL https://arxiv.org/abs/2507.23361.

[40] Yi-Xuan Deng, Xiaoqin Liu, Yi Zhang, Guo-Wei Yang, and Shuojin Yang. Your code agent can grow alongside you with structured memory, 2026. URL https://arxiv.org/abs/2603.13258.

[41] Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents, 2026. URL https://arxiv.org/abs/2604.14004.

[42] Aditya Aggarwal and Nahid Farhady Ghalaty. Self-improving AI coding agents through accumulated behavioral rules: A closed-loop framework, 2026. URL https://arxiv.org/abs/2607.13091.

[43] Haichuan Hu, Guoqing Xie, Quanjun Zhang, Jiawei Liu, Shengcheng Yu, Chunrong Fang, Zhenyu Chen, and Liang Xiao. Evorepair: Enhancing vulnerability repair agents through experience-based self-evolution, 2026. URL https://arxiv.org/abs/2605.30105.

[44] Huawei Lin, Peng Li, Jie Song, Fuxin Jiang, and Tieying Zhang. MUSE-Autoskill: Self-evolving agents via skill creation, memory, management, and evaluation, 2026. URL https://arxiv.org/abs/2605.27366.

[45] Yanzhou Li, Yiran Zhang, Xiaoyu Zhang, Xiaoxia Liu, and Yang Liu. CODESKILL: Learning self-evolving skills for coding agents, 2026. URL https://arxiv.org/abs/2605.25430.

[46] Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. Skillforge: Forging domain-specific, self-evolving agent skills in cloud technical support. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 4763–4768. ACM, 2026. doi: 10.1145/3805712.3808466. Industry Track.

[47] Shangyin Tan, Lakshya A. Agrawal, Rohit Sandadi, Dan Klein, Koushik Sen, Alexandros G. Dimakis, and Matei Zaharia. Automatically learning skills for coding agents. In *Proceedings of the ACM Conference on AI and Agentic Systems*, 2026. doi: 10.1145/3786335.3813196. URL https://doi.org/10.1145/3786335.3813196.

[48] Chen Yang, Jiashuo Tian, Ziqi Wang, Xinyin Liu, Meiru Ye, and Junjie Chen. Learning globally reusable skills for coding agents, 2026. URL https://arxiv.org/abs/2608.06153.

[49] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, Rabat, Morocco, March 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.eacl-long.333. URL https://aclanthology.org/2026.eacl-long.333/.

[50] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=eC4ygDs02R.

[51] Yifan Yang, Ziyang Gong, Weiquan Huang, Qihao Yang, Ziwei Zhou, Zisu Huang, Yan Li, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Yuqing Yang, Dongdong Chen, Xue Yang, and Chong Luo. Skillopt: Executive strategy for self-evolving agent skills, 2026. Preprint, arXiv:2605.23904.

[52] Yifei Shen, Bo Li, and Xinjie Zhang. Skillopt-lite: Better and faster agent self-evolution via one line of vibe, 2026. Preprint, arXiv:2607.03451.

[53] Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. Meta context engineering via agentic skill evolution. In *Proceedings of the 43rd International Conference on Machine Learning (ICML 2026)*, 2026. URL https://icml.cc/virtual/2026/poster/64296. arXiv:2601.21557.

[54] Yuetian Du, Yucheng Wang, He Xu, Jiexu Xu, Shanwen Tan, Bing Zhao, Boyu Yang, Zhijie Xu, Ming Kong, Hu Wei, Jie Liu, and Qiang Zhu. Living-harness is an interactive-agent evolver, 2026. URL https://arxiv.org/abs/2607.26598.

[55] Yifan Wang, Hao Cheng, Xiaomin Li, Yuexing Hao, Hemanth Neelgund Ramesh, Dongwon Jung, Hao Tang, Keru Wang, Chenliang Zhou, Qianhui Wu, Wenlin Yao, Ananth Grama, Andrzej Banburski-Fahey, Baolin Peng, Jaron Lanier, and Jianfeng Gao. Certified long-horizon code agent evolution via validation-gated skill optimization, 2026. URL https://arxiv.org/abs/2609.32990.

[56] Jincheng Ren, Siwei Wu, Yizhi Li, Kang Zhu, Shu Xu, Boyu Feng, Ruibin Yuan, Wei Zhang, Riza Batista-Navarro, Jian Yang, and Chenghua Lin. A self-evolving framework for efficient terminal agents via observational context compression, 2026. URL https://arxiv.org/abs/2604.19572.

[57] Jingzhi Gong, Ruizhen Gu, Zhiwei Fei, Yazhuo Cao, Lukas Twist, Alina Geiger, Shuo Han, Dominik Sobania, Federica Sarro, and Jie M. Zhang. SkillMOO: Multi-objective optimization of agent skills for software engineering, 2026. URL https://arxiv.org/abs/2604.09297.

[58] Zimu Wang, Yuling Shi, Mengfan Li, Zijun Liu, Jie M. Zhang, Chengcheng Wan, and Xiaodong Gu. EffiSkill: Agent skill based automated code efficiency optimization, 2026. URL https://arxiv.org/abs/2603.27850.

[59] Yunhao Yang, Neel P. Bhatt, Kevin Wang, Samuel Tetteh, Zhangyang Wang, and Ufuk Topcu. VASO: Formally verifiable self-evolving skills for physical AI agents, 2026. URL https://arxiv.org/abs/2606.05395.

[60] Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. MemEvolve: Meta-evolution of agent memory systems, 2025. URL https://arxiv.org/abs/2512.18746.

[61] Zihan Chen, Songwei Dong, Chengshuai Shi, Peng Wang, Song Wang, Cong Shen, and Jundong Li. The past is prologue: A plug-in controller for selective updates in sequentially evolving llm memory, 2026. URL https://arxiv.org/abs/2606.31121. arXiv preprint.

[62] Zisu Huang, Jingwen Xu, Yifan Yang, Ziyang Gong, Qihao Yang, Muzhao Tian, Xiaohua Wang, Changze Lv, Xuemei Gao, Qi Dai, Bei Liu, Kai Qiu, Xue Yang, Dongdong Chen, Xiaoqing Zheng, and Chong Luo. From raw experience to skill consumption: A systematic study of model-generated agent skills, 2026. URL https://arxiv.org/abs/2605.23899.

[63] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026. doi: 10.18653/v1/2026.acl-long.27. URL https://aclanthology.org/2026.acl-long.27/.

[64] Anthropic. How claude remembers your project. https://code.claude.com/docs/en/memory, 2026. Claude Code documentation.

[65] Google. Gemini CLI Auto Memory. https://github.com/google-gemini/gemini-cli/blob/main/docs/cli/auto-memory.md, 2026.

[66] GitHub. About GitHub Copilot Memory. https://docs.github.com/en/copilot/concepts/agents/copilot-memory, 2026.

[67] Anthropic. Equipping agents for the real world with agent skills. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills, 2025.

[68] Xijia Tao, Yihua Teng, Xinyu Fu, Ziru Liu, Kecheng Chen, Yuzhi Zhao, Suiyun Zhang, Rui Liu, and Lingpeng Kong. Softskill: Behavioral compression for contextual adaptation, 2026. URL https://arxiv.org/abs/2606.20333.

[69] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, Qunhong Zeng, Di Wang, Yuanli Wang, Roey Ben Chaim, Penghao Jiang, Haotian Shen, Luyang Kong, Xinyi Liu, Runhui Wang, Xuanqing Liu, Jiachen Li, Xin Lan, Yueqian Lin, Wengao Ye, Junwei He, Songlin Li, Yue Zhang, Yipeng Gao, Yijiang Li, Ze Ma, Liqiang Jing, Tianyu Wang, Kaixin Li, Yiqi Xue, Haoran Lyu, Yizhuo He, Yuchen Tian, Shutong Wu, Bowei Wang, Yixuan Gao, Bo Chen, Litong Liu, Sikai Cheng, Jiajun Bao, Shuaicheng Tong, Shuwen Xu, Terry Yue Zhuo, Tinghan Ye, Qi Qi, Miao Li, Longtai Liao, Zelin Tan, Chang Shi, Xilin Tang, Srinath Tankasala, Boqin Yuan, Yaoyao Qian, Jianhong Tu, Chenguang Wang, Yizhou Sun, Wei Wang, Aaron Taylor, Ziyue Yang, Changkun Guan, Zhikang Dong, Xinyu Zhang, Steven Dillmann, Han chung Lee, and Dawn Song. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026. URL https://arxiv.org/abs/2602.12670. arXiv preprint.

[70] Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents, 2026. arXiv:2605.30621.

[71] Soham Gadgil, David Alexander, Sai Sunku, and Franziska Roesner. Bad memory: Evaluating prompt injection risks from memory in agentic systems, 2026. URL https://arxiv.org/abs/2607.14611. arXiv preprint.

[72] Pritam Dash, Tongyu Ge, Aditi Jain, Tanmay Shah, and Zhiwei Shang. From untrusted input to trusted memory: A systematic study of memory poisoning attacks in llm agents, 2026. URL https://arxiv.org/abs/2606.04329. arXiv preprint.

[73] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 62743–62767. PMLR, 2024.

[74] Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Sercan Ö. Arık. Multi-agent design: Optimizing agents with better prompts and topologies. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/1ab4e0e8f35078109bc78d7b465d306f-Abstract-Conference.html. Originally released as arXiv:2502.02533.

[75] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=PFS4ffN9Yx.

[76] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*,

pages 75834–75852. PMLR, PMLR, 2025. URL https://proceedings.mlr.press/v267/zhang25bi.html.

[77] Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM agent search in modular design space. In *The Thirteenth International Conference on Learning Representations*, Singapore, 2025. URL https://openreview.net/forum?id=mPdmDYIQ7f.

[78] Yixin Ou, Wangchunshu Zhou, Shengwei Ding, Long Li, Jialong Wu, Tiannan Wang, Jiamin Chen, Shuai Wang, Xiaohua Xu, Ningyu Zhang, Huajun Chen, and Yuchen Eleanor Jiang. Symbolic learning enables self-evolving agents. *AI Open*, 6:314–322, 2025. doi: 10.1016/j.aiopen.2025.11.004. Originally released as arXiv:2406.18532.

[79] Siwei Liu, Jinyuan Fang, Han Zhou, Yingxu Wang, and Zaiqiao Meng. SEW: Self-evolving agentic workflows for automated code generation, 2025. URL https://arxiv.org/abs/2505.18646.

[80] Zihan Ma, Zhikai Zhao, Chuanbo Hua, Federico Berto, and Jinkyoo Park. JudgeFlow: Agentic workflow optimization via block judge, 2026. URL https://arxiv.org/abs/2601.07477.

[81] Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026. arXiv preprint.

[82] Zhezheng Hao, Tianfu Wang, Huanshuo Dong, Ziyan Liu, Hong Wang, Xiankun Lin, Qiang Lin, Can Wang, Hande Dong, and Jiawei Chen. Evolve as a team: Collaborative self-evolution for LLM-based multi-agent systems, 2026. URL https://arxiv.org/abs/2605.29790.

[83] Simon Yu, Gang Li, Weiyan Shi, and Peng Qi. Polyskill: Learning generalizable skills through polymorphic abstraction, 2025. URL https://arxiv.org/abs/2510.15863.

[84] Zeyu Ren, Ling Yue, Ran Li, Yishu Wang, Shengxiang Xu, Hanmo Liu, Shaowu Pan, and Shimin Di. FlowEvo: Self-evolving agents through the co-evolution of workflows and executable skills, 2026. URL https://arxiv.org/abs/2607.21596.

[85] Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. Live-SWE-agent: Can software engineering agents self-evolve on the fly?, 2025. URL https://arxiv.org/abs/2511.13646.

[86] Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves, 2026. Preprint, arXiv:2606.09498.

[87] Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws, 2026. URL https://arxiv.org/abs/2606.06324.

[88] Luan Zhang, Ruochen Zhou, Dandan Song, Zhengyu Chen, Yuhang Tian, Jun Yang, Huipeng Ma, Chenhao Li, Guangyuan Feng, Xudong Li, Yizhou Jin, and Yan Xu. Harnesscompass: Guiding automatic harness evolution toward generalizable and effective agent harnesses, 2026. URL https://arxiv.org/abs/2608.01918.

[89] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. arXiv:2604.25850.

[90] Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. AutoSaddler: Automatic harness optimization with durable updates from agent execution traces, 2026. URL https://arxiv.org/abs/2608.23041.

[91] Tianxin Wei, Zhan Shi, Minhua Lin, Bing He, Zewen Liu, Yisi Sang, Yuanchen Bei, Xuying Ning, Jiaru Zou, Ting-Wei Li, Xiao Lin, Yanjun Zhao, Chi Wang, Benoit Dumoulin, Dakuo Wang, Jingrui He, and Hanqing Lu. Evo-Harness: Context-to-harness skill compilation for self-evolving agents, 2026. URL https://arxiv.org/abs/2608.15071.

[92] Guojun Zhu, Xunheng Huang, Peng Yin, Jiahui Xie, Sanguo Zhang, and Doudou Zhou. Bad genius: Counterfactual-guided harness evolution beyond task-specific shortcuts, 2026. URL https://arxiv.org/abs/2609.18366.

[93] Ruiqing Yue, Yu Cui, Zhuoyu Sun, Sicheng Pan, Xianhong Xue, Tingyu Li, Ting Li, Wenzhuo Zhu, Yi Chen, Yifei Liu, Baohan Huang, Zhe Cui, Haibin Zhang, and Cong Zuo. Ecdysis: Efficient and effective training of runtime harnesses for llm agents, 2026. URL https://arxiv.org/abs/2609.11677.

[94] Geyi Yang, Zikun Qu, Xiang Li, Zhiyong Wang, Min Zhang, Shipei Zeng, and Zhongxiang Dai. GUI-HARVEST: Self-improving GUI agents through evidence-driven harness evolution, 2026. URL https://arxiv.org/abs/2610.00948.

[95] Yuan Tian, Bing Hu, Hao Wang, Binghang Lu, and Fang Wu. Harness evolution hits a ceiling: When weight training should begin, 2026. URL https://arxiv.org/abs/2610.11655.

[96] Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. Harnessbank: Semantic gene-bank search with gated verification for agent-harness self-evolution, 2026. URL https://arxiv.org/abs/2607.13683.

[97] Zhezheng Hao, Hong Wang, Jian Luo, Jianqing Zhang, Yuyan Zhou, Qiang Lin, Can Wang, Hande Dong, and Jiawei Chen. ReCreate: Reasoning and creating domain agents driven by experience. https://aclanthology.org/2026.acl-long.1432/, 2026.

[98] Andrew Borthwick and Stephen Ash. RoboPhD: Self-improving text-to-SQL through autonomous agent evolution, 2026. URL https://arxiv.org/abs/2601.01126.

[99] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*, Singapore, 2025. URL https://openreview.net/forum?id=t9U3LW7JVX.

[100] Yoonho Lee, Roshen S. Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. In *Third Conference on Language Modeling (COLM 2026)*, 2026. Originally released as arXiv:2603.28052.

[101] Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. Rrsi: Regularized recursive self-improvement of agent harnesses, 2026. URL https://arxiv.org/abs/2609.24972.

[102] Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=pUpzQZTvGY. Poster.

[103] Yifan Zhang, Yutong Dai, Juntao Tan, Luyu Yang, Rishi Mullur, Thai Hoang, Zhiyuan Hu, James Zhu, Phil Mui, Silvio Savarese, Ran Xu, and Zeyuan Chen. Darwinx: Evolving agent harnesses through natural selection, 2026. URL https://arxiv.org/abs/2608.07545. arXiv preprint.

[104] Wenyi Wang, Piotr Piękos, Li Nanbo, Firas Laakom, Yimeng Chen, Mateusz Ostaszewski, Mingchen Zhuge, and Jürgen Schmidhuber. Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine, 2025. URL https://arxiv.org/abs/2510.21614.

[105] Zhaotian Weng, Antonis Antoniades, Deepak Nathani, Zhen Zhang, Xiao Pu, and Xin Eric Wang. Group-Evolving Agents: Open-ended self-improvement via experience sharing, 2026. URL https://arxiv.org/abs/2602.04837.

[106] Di Zhang. AgentDevel: Reframing self-evolving LLM agents as release engineering, 2026. URL https://arxiv.org/abs/2601.04620.

[107] Anton Razzhigaev, Andrei Gritsaev, Andrei Kaznacheev, Nikita Dragunov, Roman Yampolskiy, and Andrei Kuznetsov. Ouroboros: A self-developing frontier coding agent with reviewed core evolution, 2026. arXiv:2608.08311.

[108] Qianshu Cai, Yonggang Zhang, Xianzhang Jia, Huajiang Zheng, Wei Xue, Jun Song, Xinmei Tian, and Yike Guo. Moss: Self-evolution through source-level rewriting in autonomous agent systems, 2026. URL https://arxiv.org/abs/2605.22794. arXiv preprint.

[109] Jun Nie, Yonggang Zhang, Qianshu Cai, Yiu ming Cheung, Xinmei Tian, and Bo Han. EvolveNet: Collaborative harness evolution for agent self-improvement, 2026. URL https://arxiv.org/abs/2608.04968.

[110] Zekai Wang, Yingqiang Ge, Zekun Wang, Hai Wang, Yuhui Xu, Joshua Frandsen, Shancong Fu, Ashia C. Wilson, and Chandan K. Reddy. VERSE: Verified self-evolving optimizer for agent harnesses, 2026. URL https://arxiv.org/abs/2610.02616.

[111] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.1354. URL https://aclanthology.org/2025.acl-long.1354/.

[112] Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution, 2026. URL https://arxiv.org/abs/2607.08124.

[113] Tianshi Xu, Huifeng Wen, and Meng Li. Adapting the interface, not the model: Runtime harness adaptation for deterministic LLM agents, 2026. URL https://arxiv.org/abs/2605.22166.

[114] Haoyang Yan, Min le Su, Hangfan Zhang, Zhanhao Li, Chen Zhang, Shao Zhang, Yang Chen, Lei Bai, and Shuyue Hu. Harness-of-Harness: Multi-day autonomous software development with continual improvement, 2026. URL https://arxiv.org/abs/2609.01481.

[115] Erik Schluntz and Barry Zhang. Building effective AI agents. https://www.anthropic.com/engineering/building-effective-agents, 2024. Anthropic engineering blog, December 2024.

[116] Yifan Shi, Wei Zhang, and Tianyi Cui. A programming paradigm for spatiotemporal composability, 2026. arXiv:2608.25512.

[117] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026. URL https://arxiv.org/abs/2606.14249. arXiv preprint.

[118] Boxiu Li, Zimo Wen, Yijia Fan, Chuan Wen, Fan Yang, Hangxi Guo, Jiaao Wu, Jiachen Zhang, Junxiang Lei, Mukai Li, Ruize Tang, Runjing Gu, Shibo Hu, Sihan Chen, Sufeng Guo, Wanbo Zhang, Xian Zhang, Xiaoyu Chen, Xuanhe Zhou, Xuyao Huang, Yifei Gao, Yifei Shen, Yilin Chen, Yuheng Wu, Yuzhe Zhang, Zelong Zhao, and Zhijie Deng. Argus: A general-purpose agentic reasoning runtime for long-horizon tasks, 2026. URL https://arxiv.org/abs/2608.05144.

[119] Wentao Zhang, Zhe Zhao, Haibin Wen, Yingcheng Wu, Cankun Guo, Ming Yin, and Bo An. Autogenesis: A self-evolving agent protocol, 2026. URL https://arxiv.org/abs/2604.15034.

[120] Siwei Wu, Jincheng Ren, Yizhi Li, Haau-Sing Li, Chengran Yang, Yuxuan Zhang, Weicheng Gu, Jian Yang, Riza Batista-Navarro, Chuanyi Zhang, Xianglong Liu, Ming Zhou, Bryan Dai, and Chenghua Lin. Modularrsi: Modular and generalizable recursive harness self-improvement, 2026. URL https://arxiv.org/abs/2609.14857.

[121] Zeyu Gan, Zixuan Gong, and Yong Liu. Harness evolution as learning: Approximation, generalization, and optimization limits of self-improving personal agents, 2026. URL https://arxiv.org/abs/2609.36892.

[122] Qianshu Cai, Yonggang Zhang, Jun Nie, Maohao Ran, Huajiang Zheng, Jun Song, Xinmei Tian, Yike Guo, and Wei Xue. A theory of reliable self-evolution for agent harnesses, 2026. URL https://arxiv.org/abs/2609.08175.

[123] Xi Chen, Christos Papadimitriou, and Binghui Peng. Memory bounds for continual learning. In *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022. doi: 10.1109/FOCS54457.2022.00056.

[124] Shai Shalev-Shwartz, Ohad Shamir, Nathan Srebro, and Karthik Sridharan. Learnability, stability and uniform convergence. *Journal of Machine Learning Research*, 11:2635–2670, 2010.

[125] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[126] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. Generalization in adaptive data analysis and holdout reuse. *arXiv preprint arXiv:1506.02629*, 2015.

[127] Oussama Ben Sghaier, Hao Li, Bram Adams, and Ahmed E. Hassan. Don't blame the large language model: How agent harness evolution shapes coding agent quality, 2026. URL https://arxiv.org/abs/2607.03691.

[128] Naman Vats and Oleg Golev. The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation, 2026. URL https://arxiv.org/abs/2607.22585.

[129] Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents, 2026. URL https://arxiv.org/abs/2607.12227.

[130] Varun Ursekar, Apaar Shanker, Yash Maurya, Shehab Yasser, Vijay S. Kalmath, Veronica Chatrath, and Yuan Xue. Harnessopt-bench: Evaluating llms at harness optimization, 2026. URL https://arxiv.org/abs/2608.06301.

[131] Yuhao Wu, Jingyuan Zhang, Jiajun Shi, Xinping Lei, Qingshui Gu, Yuxuan Zhang, Zexuan Wang, Chen He, Chen Huang, Maojia Song, Zhiyuan Zeng, Shaowen Wang, Jinkai Liu, Yunfeng Shi, Jiaheng Liu, Shen Yan, Wenhao Huang, Ge Zhang, and Wenxuan Zhang. Harnessdev: Can llms create and evolve their own agent harness?, 2026. URL https://arxiv.org/abs/2609.01437.

[132] Leijun Zhou, Zhihao Liu, Xiang Qu, Chenxu Liu, Yifei Liu, Yanke Yu, Jingzhe Xu, Xuejun Wu, Buyue Qian, Xi Chen, Yaowei Zheng, and Junhao Hu. GDPevo: Evaluating agent self-evolution on real business tasks, 2026. URL https://arxiv.org/abs/2608.03764.

[133] Zixuan Ke, Vaidehi Patil, Haizhou Shi, Yang Li, Ye Liu, Sarath Shekkizhar, Anurag Koul, Jiayu Wang, Xuan Phi Nguyen, Semih Yavuz, Mohit Bansal, and Shafiq Joty. Evoharnessbench: Can your agents keep pace with an evolving harness?, 2026. URL https://arxiv.org/abs/2609.04280.

[134] Jiajie Wang, Yutong Zhao, Tianlin Li, Huashan Chen, Jinfu Chen, Kebin Peng, and Sen He. Agent skill evolution: How revisions affect coding agents, 2026. URL https://arxiv.org/abs/2610.04832.

[135] Gavin C Cawley and Nicola LC Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *The Journal of Machine Learning Research*, 11: 2079–2107, 2010.

[136] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349 (6248):636–638, 2015.

[137] Su Wang, Pin Qian, Yifan Lin, Jingzhou Xu, Yihang Chen, Xiaochong Jiang, Lifei Liu, and Haoran Yu. Phantom Guardrails: When self-improving agent harnesses fix failures that never happened, 2026. URL https://arxiv.org/abs/2607.13083.

[138] Xing Wang, Xiaoyi Zhang, and Jie Shao. Auditing harness tampering in self-improving agents, 2026. URL https://arxiv.org/abs/2609.00069.

[139] Diandian Guo, Cong Cao, Fangfang Yuan, Yingqi Wang, Yueshan Wang, and Dakui Wang. Self-authored verification is unreliable in heuristic self-improving agents, 2026. URL https://arxiv.org/abs/2607.24300.

[140] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. SWE-Bench+: Enhanced coding benchmark for LLMs, 2024. URL https://arxiv.org/abs/2410.06992.

[141] Michael Nguyen, Quoc Nguyen, and Paul Vuong. Recursive self-evolving agents via held-out selection, 2026. URL https://arxiv.org/abs/2606.28374.

[142] Biswa Sengupta. Self-evolving agents with anytime-valid certificates, 2026. URL https://arxiv.org/abs/2607.00871.

[143] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents, 2025. URL https://arxiv.org/abs/2505.20411.

[144] Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, Elsie Nallipogu, Qingwei Lin, Yingnong Dang, Saravan Rajmohan, and Dongmei Zhang. SWE-bench goes live!, 2025. URL https://arxiv.org/abs/2505.23419.

[145] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. Ai agents that matter. *Transactions on Machine Learning Research*, 2025.

[146] Longju Bai, Zhemin Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do AI agents spend your money? analyzing and predicting token consumption in agentic coding tasks, 2026. URL https://arxiv.org/abs/2604.22750.

[147] Mohammad Asadolahi, Amir Amini, Samira Talebi, Amirfarhad Farhadi, and Azadeh Zamanifar. Memory reward inflation in self-improving LLM agents, 2026. URL https://arxiv.org/abs/2608.00017.

[148] Xing Zhang, Guanghui Wang, Yanwei Cui, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Who grades the grader? co-evolving evaluation metrics and skills for self-improving LLM agents, 2026. URL https://arxiv.org/abs/2607.12790.

[149] Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. Agentrewardbench: Evaluating automatic evaluations of web agent trajectories. In *Second Conference on Language Modeling (COLM 2025)*, 2025. URL https://colm.cc/virtual/2025/poster/571. Originally released as arXiv:2504.08942.

[150] Jane Pan, He He, Samuel R. Bowman, and Shi Feng. Spontaneous reward hacking in iterative self-refinement, 2024. URL https://arxiv.org/abs/2407.04549.

[151] Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, Qi Liu, Zhifang Sui, and Tong Yang. Claw-eval: Towards trustworthy evaluation of autonomous agents, 2026. URL https://arxiv.org/abs/2604.06132. arXiv preprint.

[152] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S. Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, Franck Ndzomga, Dheeraj Oruganty, Sophie Luskin, Kangheng Liu, Botao Yu, Amit Arora, Dongyoon Hahm, Harsh Trivedi, Huan Sun, Juyong Lee, Tengjun Jin, Yifan Mai, Yifei Zhou, Yuxuan Zhu, Rishi Bommasani, Daniel Kang, Dawn Song, Peter Henderson, Yu Su, Percy Liang, and Arvind Narayanan. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/a0928f924a344aaebbb7f6cd8d56e34c-Paper-Conference.pdf.

[153] Xidong Yang, Xingyi Zhang, Wenhao Li, Wenyan Liu, Junjie Sheng, Yun Hua, Wei Yin, Tao Fang, Chuyun Shen, and Xiangfeng Wang. Path-bench: Path-dependent evaluation of lifelong agents, 2026. URL https://arxiv.org/abs/2608.01149. arXiv preprint.

[154] Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, Zhongzhi Li, Yingying Zhang, Le Song, and Qianli Ma. LifelongAgentBench: Evaluating LLM agents as lifelong learners, 2025. URL https://arxiv.org/abs/2505.11942.

[155] Tianyi Guan, Yiding Wang, Haotong Yang, Siyuan Cao, Shirui Liu, Yi Hu, Jiaqi Li, and Muhan Zhang. ContinualSkillBench: Can LLM agents truly evolve their capabilities?, 2026. URL https://arxiv.org/abs/2608.03874.

[156] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. SkillLearnBench: Benchmarking continual learning methods for agent skill generation on real-world tasks, 2026. URL https://arxiv.org/abs/2604.20087.

[157] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks, 2026. URL https://arxiv.org/abs/2602.16313.

[158] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-Memory: Benchmarking LLM agent test-time learning with self-evolving memory, 2025. URL https://arxiv.org/abs/2511.20857.

[159] Haizhong Zheng, Yizhuo Di, Ranajoy Sadhukhan, Shuowei Jin, and Beidi Chen. ServeLearnBench: How well can agents self-improve from serving experience?, 2026. URL https://arxiv.org/abs/2610.07792.

[160] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Ratchet: How reliable must an LLM judge be to retire a skill?, 2026. URL https://arxiv.org/abs/2605.22148.

[161] Hongyi Liu, Haoyan Yang, Tao Jiang, Bo Tang, Feiyu Xiong, Yuyu Luo, and Zhiyu Li. SkillsVote: Lifecycle governance of agent skills from collection, recommendation to evolution, 2026. URL https://arxiv.org/abs/2605.18401.

[162] Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. Library Drift: Diagnosing and fixing a silent failure mode in self-evolving LLM skill libraries, 2026. URL https://arxiv.org/abs/2605.19576.

[163] Linfang Shang, Ming Xu, Yiding Sun, Tianle Xia, Lingxiang Hu, Lan Xu, and Ning Zheng. When self-evolution backfires: Pre-commit gating against skill contamination in LLM agents, 2026. URL https://arxiv.org/abs/2608.05810.

[164] Xiaodong Wu, Yu Shi, Qi Li, Zhimin Zhao, Xiangman Li, Bram Adams, Ahmed E. Hassan, and Jianbing Ni. EVOMAL: Self-poisoning in self-evolving coding agents, 2026. URL https://arxiv.org/abs/2608.25776.

[165] Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Haibo Qiu, Qingnan Ren, Shun Zou, Zehui Chen, Feng Zhao, et al. SkillFlow: Benchmarking lifelong skill discovery and evolution for autonomous agents, 2026. URL https://arxiv.org/abs/2604.17308.

[166] Bowei Xia, Mengkang Hu, Shijian Wang, Jiarui Jin, Wenxiang Jiao, Yuan Lu, Kexin Li, and Ping Luo. Tool-Genesis: A task-driven tool creation benchmark for self-evolving language agent, 2026. URL https://arxiv.org/abs/2603.05578.

[167] Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Shisong Chen, Jinghao Zhang, Tianjun Pan, Weijia Zhou, Jiaqing Liang, and Yanghua Xiao. SEA-Eval: A benchmark for evaluating self-evolving agents beyond episodic assessment, 2026. URL https://arxiv.org/abs/2604.08988.

[168] Tue Le, Minh V. T. Thai, Thong T. Nguyen, Van Nguyen, Thanh Le, Anh Nguyen, and Trung Le. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios, 2025. URL https://arxiv.org/abs/2512.18470.

[169] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, and Joseph E. Gonzalez. Continual learning bench: Evaluating frontier AI systems in real-world stateful environments, 2026. URL https://arxiv.org/abs/2606.05661.

[170] Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, Yuxin Chen, Mengdi Wang, and Ling Yang. PAST-Bench: Benchmarking the foundations of recursive self-improvement in personal agents, 2026. URL https://arxiv.org/abs/2608.04003.

[171] Bo Deng, Kang Zhou, Lifan Guo, Chongyang Tao, Xuanren Chen, Chenggang Xie, Renzhao Liang, Feng Chen, and Chi Zhang. FinEvo-Bench: A longitudinal benchmark for self-evolving agents in professional financial workflows, 2026. URL https://arxiv.org/abs/2608.06144.

[172] Harman Singh, Anton Bakhtin, Rulin Shao, Gabriel Synnaeve, Ilia Kulikov, Rob Fergus, Sanjeev Arora, Kurt Keutzer, Jason Weston, Anuj Mahajan, and Anirudh Goyal. Agent plasticity: Measuring self-improvement through experience, 2026. URL https://arxiv.org/abs/2610.08902.

[173] Xihao Piao, Zifeng Wang, and Zhen Chen. Do self-evolving skills generalize to held-out tasks?, 2026. URL https://arxiv.org/abs/2609.39148.

[174] Siyuan Wang, Zhuohan Long, Zhihao Fan, Zhongyu Wei, and Xuanjing Huang. Benchmark self-evolving: A multi-agent framework for dynamic LLM evaluation, 2024. URL https://arxiv.org/abs/2402.11443.

[175] Yongrui Chen, Yangyang Ma, Xiaoying Huang, Shenyu Zhang, Huajun Chen, Haofen Wang, and Guilin Qi. StressEval: Failure-driven dynamic benchmarking for knowledge-intensive reasoning in large language models, 2026. URL https://arxiv.org/abs/2605.01939.

[176] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-limited LLM benchmark, 2024. URL https://arxiv.org/abs/2406.19314. ICLR 2025 Spotlight.

[177] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

[178] Zhiling Yan, Dingjie Song, Zhe Fang, Yisheng Ji, Xiang Li, Quanzheng Li, and Lichao Sun. LiveMedBench: A contamination-free medical benchmark for LLMs with automated rubric evaluation, 2026. URL https://arxiv.org/abs/2602.10367.

[179] Zhiqiu Xu, Shibo Jin, Shreya Arya, and Mayur Naik. MathDuels: A self-play benchmark that grows, 2026. URL https://arxiv.org/abs/2604.21916. COLM 2026.

[180] Alex Iacob, Andrej Jovanović, William F. Shen, Daniel Burkhardt, Meghdad Kurmanji, Nurbek Tastan, Lorenzo Sani, Niccolò Alberto Elia Venanzi, Ambroise Odonnat, Zeyu Cao, Bill Marino, Xinchi Qiu, and Nicholas D. Lane. The red queen gödel machine: Co-evolving agents and their evaluators, 2026. URL https://arxiv.org/abs/2606.26294.

[181] Cognition. Cascade Memories. https://docs.devin.ai/desktop/cascade/memories, 2026.

[182] Cursor. Automations. https://cursor.com/changelog/03-05-26, 2026. Cursor Changelog, March 5, 2026.

[183] Augment Code. How We Built Memory Review. https://www.augmentcode.com/blog/how-we-built-memory-review, 2025.

[184] Cognition. Devin Session Insights. https://docs.devin.ai/product-guides/session-insights, 2026.

[185] Replit. Replit Agent Skills. https://docs.replit.com/features/agent/skills, 2026.

[186] Letta. Introducing the Letta Code App. https://www.letta.com/blog/introducing-the-letta-code-app/, 2026.

[187] OpenClaw. OpenClaw Skills. https://docs.openclaw.ai/tools/skills, 2026.

[188] Nous Research. Hermes Agent: Skills System. https://hermes-agent.nousresearch.com/docs/user-guide/features/skills, 2026.

[189] Prime Intellect. Prime Agent: A Self-Improving RLM Harness. https://github.com/PrimeIntellect-ai/prime-agent, 2026.

[190] Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, Maohao Shen, Vishy Tirumalashetty, George Lee, Jiawei Han, Tomas Pfister, and Chen-Yu Lee. Skillos: Learning skill curation for self-evolving agents, 2026. URL https://arxiv.org/abs/2605.06614. arXiv preprint.

[191] Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks. In *Second Conference on Language Modeling (COLM 2025)*, 2025. URL https://colm.cc/virtual/2025/poster/533. Originally released as arXiv:2504.06821.

[192] Doyun Kim, Chanwoo Kim, Sugyeong Eo, Yeo-Chan Yoon, and Chanjun Park. Evoskill injection: Red-teaming autonomous skill generation and evolution in self-evolving agents, 2026. Accepted to EMNLP 2026 (per the arXiv comments field); arXiv:2608.30429.

[193] Joar Skalse, Nikolaus Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward gaming. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, volume 35, pages 9460–9471. Curran Associates, Inc., 2022. URL https://papers.nips.cc/paper_files/paper/2022/hash/3d719fee332caa23d5038b8a90e81796-Abstract-Conference.html. Originally released as arXiv:2209.13085 under the title "Defining and Characterizing Reward Hacking".

[194] Ömer Veysel Çağatan and Xuandong Zhao. Reward hacking in language model agents: Revisiting AI safety gridworlds, 2026. URL https://arxiv.org/abs/2606.15385. arXiv preprint.

[195] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL https://arxiv.org/abs/2504.08066.

[196] Ao Qu, Han Zheng, Zijian Zhou, Yihao Yan, Yihong Tang, Shao Yong Ong, Fenglu Hong, Kaichen Zhou, Chonghe Jiang, Minwei Kong, Jiacheng Zhu, Xuan Jiang, Sirui Li, Cathy Wu, Bryan Kian Hsiang Low, Jinhua Zhao, and Paul Pu Liang. Coral: Towards autonomous multi-agent evolution for open-ended discovery. In *The Third Conference on Language Modeling (COLM 2026)*, 2026. arXiv:2604.01658.

[197] Yanlin Fei, Nazhou Liu, Xinmiao Yu, Shaolong Chen, Lei Li, Rahul Thapa, et al. How do agents fail on AutoResearch: End-to-end diagnostic evaluation on 100 real-world frontier research tasks, 2026. AutoResearchEval; arXiv preprint.

[198] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131. Google DeepMind technical report.

[199] PSI. The AI-native laboratory fueling the physics takeoff. https://psi.inc/vision/, n.d. Company vision and architecture page; accessed 22 September 2026.

[200] Dhruv Trehan and Paras Chopra. Why LLMs aren't scientists yet: Lessons from four autonomous research attempts, 2026. URL https://arxiv.org/abs/2601.03315.

[201] Sébastien Bubeck, Christian Coester, Ronen Eldan, Timothy Gowers, Yin Tat Lee, Alexandru Lupsasca, Mehtaab Sawhney, Robert Scherrer, Mark Sellke, Brian K. Spears, Derya Unutmaz, Kevin Weil, Steven Yin, and Nikita Zhivotovskiy. Early science acceleration experiments with GPT-5, 2025. URL https://arxiv.org/abs/2511.16072.

[202] Hjalmar Wijk, Tao Roa Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Joshua M. Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich, Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Jun Koba Sato, William Saunders, Maksym Taran, Ben West, and Elizabeth Barnes. RE-bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 66772–66832, 2025. URL https://proceedings.mlr.press/v267/wijk25a.html.

[203] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can LLMs write efficient GPU kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 47356–47415, 2025. URL https://proceedings.mlr.press/v267/ouyang25a.html.

[204] Konstantinos Bousmalis, Giulia Vezzani, Dushyant Rao, Coline Devin, Alex X. Lee, Maria Bauza, Todor Davchev, Yuxiang Zhou, Agrim Gupta, Akhil Raju, Antoine Laurens, Claudio

Fantacci, Valentin Dalibard, Martina Zambelli, Misha Denil, Michiel Blokzijl, Murilo Fernandes Martins, Rugile Pevceviciute, Alex Miedema, et al. Robocat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2306.11706. Originally released as arXiv:2306.11706.

[205] Tomoyuki Kagaya, Subramanian Lakshmi, Yuxuan Lou, Thong Jing Yuan, Jayashree Karlekar, Sugiri Pranata, Natsuki Murakami, Akira Kinose, and Yang You. Memory transfer planning: Llm-driven context-aware code adaptation for robot manipulation, 2025. URL https://arxiv.org/abs/2509.24160.

[206] Hyun Jung Lee, Jungtaek Kim, Jongwon Jeong, Tae-Eui Kam, Donghyun Kim, and Yong Jae Lee. EMHO: Embodied agent harness optimization via experience traces, 2026. URL https://arxiv.org/abs/2610.08432.

[207] Noematrix Team. RoboRSI: Stable, efficient, and reusable robot self-evolution in complex real-world environments. Research preview; code repository and research blog, September 2026. https://github.com/nssmd/RoboRSI; project page https://lab.noematrix.ai/blog/2-roborsi-research-preview/. Not peer-reviewed.

[208] Yuan Meng, Zhenguo Sun, Max Fest, Xiangtong Yao, Xukun Li, Zhenshan Bing, and Alois Knoll. Robotic ultra-long-horizon manipulation skills via human-guided lifelong code generation, 2025. URL https://arxiv.org/abs/2509.18597.

[209] Zhengwei Xie, Zhisheng Chen, Ziyan Weng, Jinhan Li, Chenglong Li, Zikai Xiao, Jingwei Song, Jinhao Jing, Vireo Zhang, and Kun Wang. Mineevolve: Self-evolution with accumulated knowledge for long-horizon embodied minecraft agents, 2026. URL https://arxiv.org/abs/2603.13131. arXiv preprint.

[210] Zeyu Shen, Haoxiang You, Yilang Liu, Zhicheng Zheng, Lihan Zha, Kashu Yamazaki, Mingtong Zhang, Suning Huang, Jiankai Sun, Qianzhong Chen, Lucy He, Kaiyuan Liu, Haoran Chang, Katerina Fragkiadaki, Dhruv Shah, Mac Schwager, Peter Henderson, Ian Abraham, and Canwen Xu. EmbodiedSWE: Coding Agents for Long Horizon Dexterous Robotics, 2026.

[211] Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. Agent-safetybench: Evaluating the safety of LLM agents, 2024. URL https://arxiv.org/abs/2412.14470.

[212] Aarya Doshi, Yining Hong, Congying Xu, Eunsuk Kang, Alexandros Kapravelos, and Christian Kästner. Towards verifiably safe tool use for LLM agents, 2026. URL https://arxiv.org/abs/2601.08012. arXiv preprint.

[213] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.

[214] Neel Guha, Julian Nyarko, Daniel Ho, Christopher Ré, Adam Chilton, Alex Chohlas-Wood, Austin Peters, Brandon Waldon, Daniel Rockmore, Diego Zambrano, et al. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Advances in neural information processing systems*, 36:44123–44279, 2023.

[215] Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simon Fishman, Marwan Aljubeh, Phoebe Thacker, Laurance Fauconnet, Natalie Kim,

Samuel Miserendino, Gildas Chabot, David Li, Patrick Chao, Michael Sharman, Alexandra Barr, Amelia Glaese, and Jerry Tworek. Gdpval: Evaluating AI model performance on real-world economically valuable tasks. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/290c2430f91912204f30bbcc990fff1d-Abstract-Conference.html. Originally released as arXiv:2510.04374.

[216] Yixing Jiang, Kameron C. Black, Gloria Geng, Danny Park, James Zou, Andrew Y. Ng, and Jonathan H. Chen. MedAgentBench: A realistic virtual EHR environment to benchmark medical LLM agents, 2025. URL https://arxiv.org/abs/2501.14654. arXiv preprint; associated with NEJM AI.

[217] Samuel Schmidgall, Rojin Ziaei, Carl Harris, Eduardo Reis, Jeffrey Jopling, and Michael Moor. AgentClinic: A multimodal agent benchmark to evaluate AI in simulated clinical environments, 2024. URL https://arxiv.org/abs/2405.07960. arXiv preprint; later in npj Digital Medicine.

[218] Shuai Shao, Qihan Ren, Dongrui Liu, Chen Qian, Boyi Wei, Dadi Guo, Jingyi Yang, Xinhao Song, Linfeng Zhang, Weinan Zhang, and Jing Shao. Your agent may misevolve: Emergent risks in self-evolving LLM agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/a24cd16bc361afa78e57d31d34f3d936-Abstract-Conference.html. Author order follows the ICLR proceedings record, which differs from the arXiv byline; originally released as arXiv:2509.26354.

[219] Fei Liu and Todd Mckinnon. Beyond the hype: The efficiency-throughput gap with github copilot. *Commun. ACM*, 69(10):68–77, 2026. ISSN 0001-0782. doi: 10.1145/3797488.

[220] Daron Acemoglu and Pascual Restrepo. The race between man and machine: Implications of technology for growth, factor shares, and employment. *American economic review*, 108 (6):1488–1542, 2018.

[221] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of ai on developer productivity: Evidence from github copilot. *arXiv preprint arXiv:2302.06590*, 2023.

[222] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. Reading between the lines: Modeling user behavior and costs in ai-assisted programming. In *Proceedings of the 2024 CHI conference on human factors in computing systems*, pages 1–16, 2024.

[223] Judy Hanwen Shen and Alex Tamkin. How AI impacts skill formation, 2026. URL https://arxiv.org/abs/2601.20245.

[224] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakcı, and Rei Mariman. Generative AI without guardrails can harm learning: Evidence from high school mathematics. *Proceedings of the National Academy of Sciences*, 122(26):e2422633122, 2025. doi: 10.1073/pnas.2422633122.

[225] Matthias Lehmann, Philipp B. Cornelius, and Fabian J. Sting. AI meets the classroom: When do large language models harm learning?, 2024. URL https://arxiv.org/abs/2409.09047.

[226] Greg Kestin, Kelly Miller, Anna Klales, Timothy Milbourne, and Gregorio Ponti. AI tutoring outperforms in-class active learning: an RCT introducing a novel research-based design in an authentic educational setting. *Scientific Reports*, 15:17458, 2025. doi: 10.1038/s41598-025-97652-6.

[227] Bingxi Zhao, Jiahao Zhang, Xubin Ren, Zirui Guo, Tianzhe Chu, Yi Ma, and Chao Huang. DeepTutor: Towards agentic personalized tutoring, 2026. URL https://arxiv.org/abs/2604.26962.

[228] Yang Wu, Rujing Yao, Tong Zhang, Yufei Shi, Zhuoren Jiang, Zhushan Li, and Xiaozhong Liu. Teaching according to students' aptitude: Personalized mathematics tutoring via persona-, memory-, and forgetting-aware LLMs, 2025. URL https://arxiv.org/abs/2511.15163.

[229] Unggi Lee, Minchul Shin, Yeil Jeong, Sookbun Lee, Jeongsu Moon, Kyungtae Joo, Eunjoo Lee, and Hoilym Kwon. LLMs are already good tutors: Training-free prompt optimization for pedagogical math tutoring, 2026. URL https://arxiv.org/abs/2605.27088.

[230] Aysa Xuemo Fan. Toward measuring AI's effects on skill formation: The stock-formation gap, 2026. URL https://arxiv.org/abs/2605.16283.

[231] Nicole Forsgren, Margaret-Anne Storey, Chandra Maddila, Thomas Zimmermann, Brian Houck, and Jenna Butler. The space of developer productivity: There's more to it than you think. *Queue*, 19(1):20–48, 2021.

[232] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world Github issues? In *International Conference on Learning Representations*, 2024.

[233] Bowen Baker, Joost Huizinga, Leo Gao, Zehao Dou, Melody Y. Guan, Aleksander Madry, Wojciech Zaremba, Jakub Pachocki, and David Farhi. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. URL https://arxiv.org/abs/2503.11926.

[234] Lizhi Zhang, Xin He, Dianxuan Fu, Yuyuan Feng, Jiatong Li, Qi Wang, Xin Wang, and Qinggang Zhang. SkillPoison: Progressive skill poisoning via successful experiences, 2026. URL https://arxiv.org/abs/2610.07645.

[235] Saswat Das, Parvati Viswanathan, Daniel Donnelly, Chang Huang, Sahar Abdelnabi, and Ferdinando Fioretto. SEABench: Benchmarking endogenous misalignment in self-evolving agents, 2026. URL https://arxiv.org/abs/2609.35596.

[236] Jon Kleinberg and Manish Raghavan. Algorithmic monoculture and social welfare. *Proceedings of the National Academy of Sciences*, 118(22):e2018340118, 2021. doi: 10.1073/pnas.2018340118.

[237] Rishi Bommasani, Kathleen A. Creel, Ananya Kumar, Dan Jurafsky, and Percy Liang. Picking on the same person: Does algorithmic monoculture lead to outcome homogenization? In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[238] Yi Liu, Weizhe Wang, Ruitao Feng, Yao Zhang, Guangquan Xu, Gelei Deng, Yuekang Li, and Leo Zhang. Agent skills in the wild: An empirical study of security vulnerabilities at scale, 2026. URL https://arxiv.org/abs/2601.10338.

[239] Yi Liu, Zhihao Chen, Yanjun Zhang, Gelei Deng, Yuekang Li, Jianting Ning, and Leo Yu Zhang. "Do not mention this to the user": Detecting and understanding malicious agent skills in the wild. In *35th USENIX Security Symposium (USENIX Security 2026)*, 2026. arXiv:2602.06547.

[240] Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. Beyond the protocol: Unveiling attack vectors in the model context protocol (MCP) ecosystem, 2025. URL https://arxiv.org/abs/2506.02040.

[241] Yunpeng Xiong and Ting Zhang. After the party: Growth, governance, and security scanning in the OpenClaw agent skill ecosystem. In *33rd Asia-Pacific Software Engineering Conference (APSEC 2026)*, 2026. To appear; arXiv:2609.17274.

[242] Hechuan Guo, Yongle Hao, Yue Zhang, Minghui Xu, Peizhuo Lv, Jiezhi Chen, and Xiuzhen Cheng. A measurement study of model context protocol ecosystem, 2025. URL https://arxiv.org/abs/2509.25292.

[243] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on LLM agents via query-only interaction, 2025. URL https://arxiv.org/abs/2503.03704.

[244] Stav Cohen, Ron Bitton, and Ben Nassi. Here comes the AI worm: Unleashing zero-click worms that target GenAI-powered applications, 2024. URL https://arxiv.org/abs/2403.02817.

[245] Danielle Keats Citron. Technological due process. *Wash. UL Rev.*, 85:1249, 2008.

[246] Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. SIA: Self improving AI with harness & weight updates, 2026. URL https://arxiv.org/abs/2605.27276.

[247] Jixuan Chen, Jiaxin Zhang, Qinyuan Ye, Yada Pruksachatkun, Haoxiang Zhang, Jingming Zhuo, Yifan Zhang, Yutong Dai, Juntao Tan, Xiangyu Peng, Silvio Savarese, Zeyuan Chen, Lianhui Qin, and Chien-Sheng Wu. CoTrace: Data recipes for training terminal agents with harness-model co-evolution, 2026. URL https://arxiv.org/abs/2610.10426.

[248] Yixuan Li, Yiyun Zhou, Yao Long Teng, Fuchao Yang, Yanchen Deng, Zhiyi Lyu, Xuyu Dong, Feng Chen, and Bo An. Finding the right fit: Model-harness interactions across agent tasks, 2026. URL https://arxiv.org/abs/2610.00917.

[249] Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. Shinkaevolve: Towards open-ended and sample-efficient program evolution. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/7886b9bafe76c52fd568db10ff9772df-Abstract-Conference.html. Poster; originally released as arXiv:2509.19349.

[250] Sijia Liu, Pin-Yu Chen, Bhavya Kailkhura, Gaoyuan Zhang, Alfred O. Hero, and Pramod K. Varshney. A primer on zeroth-order optimization in signal processing and machine learning: Principals, recent advances, and applications. *IEEE Signal Processing Magazine*, 37(5): 43–54, 2020.

[251] Mert Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei Zaharia, Alex Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization, 2026. arXiv:2602.20133.

[252] Avrim Blum and Moritz Hardt. The ladder: A reliable leaderboard for machine learning competitions. In *International Conference on Machine Learning*, volume 37, pages 1006–1014. PMLR, 2015.

[253] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian,

Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/444a3737adaee10d86ad2ef5f74468e6-Paper-Conference.pdf.

[254] Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason Weston, and Sainbayar Sukhbaatar. Meta-rewarding language models: Self-improving alignment with LLM-as-a-Meta-Judge, 2024.

[255] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback, 2022. Anthropic; arXiv preprint.

[256] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/8b4add8b0aa8749d80a34ca5d941c355-Abstract-Conference.html. Originally released as arXiv:2310.01798.

[257] Rui Meng, Bhavana Dalvi Mishra, Jiefeng Chen, Chun-Liang Li, Palash Goyal, Mihir Parmar, Yiwen Song, Yale Song, Rajarishi Sinha, Parthasarathy Ranganathan, Burak Gokturk, Jinsung Yoon, and Tomas Pfister. Scientistone: Towards human-level autonomous research via chain-of-evidence, 2026. URL https://arxiv.org/abs/2605.26340. arXiv preprint.